

SAPIENZA
UNIVERSITÀ DI ROMA

"SAPIENZA" UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

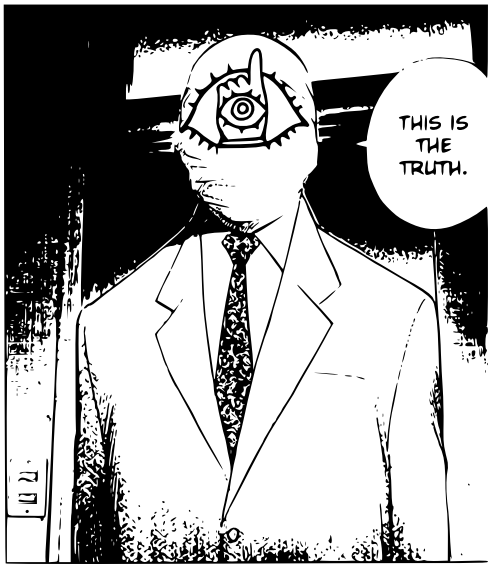
Elective in Robotics

Lecture notes

Author
Marco Casu
https://github.com/CasuFrost

October 23, 2026

This document summarizes and presents the topics for the Elective in Roboticscourse for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

I Modeling and Control of Multi-Rotor UAVs **4**

1 Introduction **5**
1.1 Quadrotor Modeling . . . 6
1.2 Quadrotor Control . . . 10
1.2.1 Linear Control . . . 11
1.2.2 Basic Control and Planning Problems . . . 12

2 Control Methods **15**
2.1 Backstepping . . . 15
2.1.1 Backstepping for Height Control . . . 17
2.1.2 PD Compared With Backstepping . . . 18
2.1.3 Integral Backstepping . . . 18
2.2 Dynamic Feedback Linearization . . . 19
2.2.1 Static Linearization . . . 20
2.2.2 Input-Output Linearization . . . 21
2.2.3 Linearization of the Quadrotor Model . . . 22
2.2.4 Considering the Complete Model . . . 25
2.2.5 Flatness . . . 27
2.3 Geometric Control on $SO(3)$ . . . 29
2.3.1 Stability of the Attitude Dynamics . . . 34
2.3.2 Position Error Dynamics . . . 35
2.3.3 Stability of the Complete System . . . 36
2.3.4 Computation of the Desired Angular Velocity and Acceleration . . . 37
2.3.5 Comparison Between DFL and Geometric Control . . . 38

3 State Estimation **40**
3.1 Sensors . . . 40
3.2 Attitude Estimation . . . 41
3.3 Velocity Estimation . . . 43
3.4 Height Estimation . . . 44
3.5 Sensor Fusion: Extended Kalman Filter . . . 44
3.5.1 Prediction Phase . . . 45
3.5.2 Update Phase . . . 45
3.5.3 State Structure and Attitude Handling . . . 45

4 Motion Planning **46**
4.1 The Motion Planning Problem . . . 46
4.2 What Is Special With Quadrotors . . . 47
4.2.1 Second Order Dynamics . . . 47
4.2.2 Underactuation . . . 48
4.2.3 Two Solution Approaches . . . 48
4.3 Flatness-Based Trajectory Generation . . . 49
4.3.1 Attitude and Thrust From the Flat Outputs . . . 49
4.3.2 Derivatives of the Angles . . . 49
4.3.3 Inputs From the Flat Outputs . . . 50
4.3.4 Feasibility of 3D Trajectories . . . 51
4.3.5 Paths With Corners . . . 51
4.4 Visibility-Based Motion Planning . . . 51
4.4.1 Visibility Roadmap Construction . . . 52
4.4.2 Search for a Safe Path . . . 53
4.4.3 Spline Interpolation . . . 54
4.4.4 Time Scaling . . . 55
4.4.5 Choice of the Yaw . . . 56
4.4.6 Example . . . 56
4.4.7 Remarks on the Visibility-Based Approach . . . 59
4.5 RRT-Based Motion Planning . . . 59
4.5.1 The Algorithm . . . 59
4.5.2 Exploration vs. Exploitation . . . 61
4.5.3 Steering With Random Inputs . . . 61
4.5.4 Control-Based Steering . . . 62
4.5.5 Remarks on the Kinodynamic RRT . . . 63
4.6 Comparison and Conclusions . . . 63

Part I

Modeling and Control of Multi-Rotor UAVs

CHAPTER

1

INTRODUCTION

UAV stands for **Unmanned Aerial Vehicle**, also known by other names depending on the domain, like *RPVs (Remotely Piloted Vehicles)* in the military domain, or *Drones* in common parlance. We will use all three names interchangeably. The methods presented are valid also for remotely operated or autonomous vehicles (at different levels of autonomy).

The term UAVs refers to a broad category of vehicles; in general, any aerial vehicle without a pilot on board is a UAV. We will examine UAVs composed of multiple rotors, in particular **quadrotors**.



Figure 1.1: Design-based classification.

Quadrotors are used in various situations:

- inspection, maintenance,
- communication networks,
- search and rescue operations,
- aerial transportation, manipulation.

These activities require a vertical, stationary and slow flight. A quadrotor is characterized by high maneuverability, can remain suspended in the air without moving (hovering), and can perform a *vertical take-off and landing (VTOL)*, unlike other vehicles such as airplanes, which require a horizontal movement in order to take off and land.

1.1 Quadrotor Modeling

We call VTOLs the vehicles that can perform a vertical take-off and landing; these are modeled with the North-East-Down (NED) reference frame:

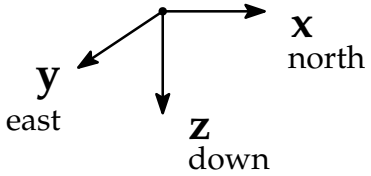


Figure 1.2: The NED reference frame.

This frame, often called SR_i, has the $\mathbf{z}$ axis pointing down to the center of the Earth. According to the dynamics of rigid bodies given by the Newton-Euler formalism, the translational dynamics of a body in the SR_i frame is given by

$$\sum_j \mathbf{F}_j = m\dot{\mathbf{V}} \tag{1.1}$$

where $\mathbf{F}_j$ are the external forces applied to the center of mass, and $\dot{\mathbf{V}}$ is the acceleration of the center of mass expressed in SR_i. The rotational dynamics is given by

$$\sum_j \mathbf{M}_j = J\dot{\mathbf{\Omega}} + \mathbf{\Omega} \times J\mathbf{\Omega} \tag{1.2}$$

where $\mathbf{M}_j$ are the external moments acting on the center of mass, $\mathbf{\Omega}$ is the angular velocity of the body expressed in the body frame, denoted SR_b, and J is the inertia tensor, also expressed in SR_b.

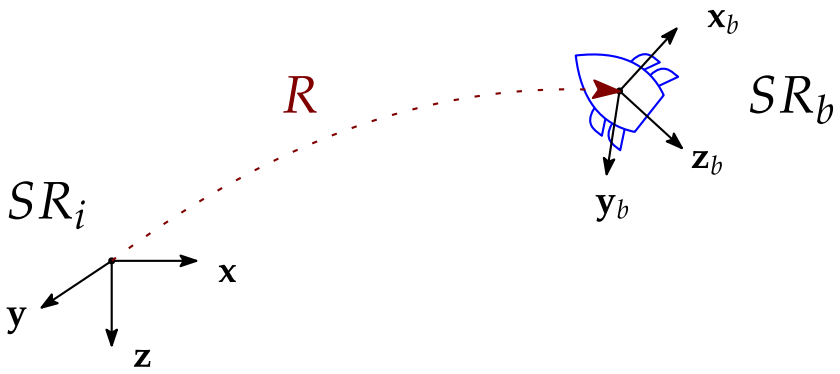


Figure 1.3: SR_b is the reference frame attached to the center of mass of the vehicle, R is the rotation matrix between the two frames.

A more explicit expression of the forces in Equation (1.1) is the following:

$$m\dot{\mathbf{V}} = -TR\mathbf{z} + \mathbf{F}_e(\mathbf{V}, \dot{\mathbf{V}}, \mathbf{\Omega}, \dot{\mathbf{\Omega}}, \mathbf{d}(t)) + R\Sigma_R\boldsymbol{\tau} + mg\mathbf{z} \tag{1.3}$$

where:

- $\mathbf{d}(t)$ are the external disturbances that do not depend on the vehicle's state.
- $\mathbf{F}_e$ are the external forces, which depend on the state.

- T is the **thrust**, which depends on the number and allocation of the rotors, while $-\mathbf{z}$ is the direction pointing upward (according to the NED frame as in Figure 1.2). R is the rotation matrix that describes the orientation of our vehicle with respect to the fixed NED frame SR_i.
- Hence, the product $-TR\mathbf{z}$ is the main upward thrust.
- $\boldsymbol{\tau}$ is the torque generated by the propellers.
- $R\Sigma_R\boldsymbol{\tau}$ are the parasitic forces generated by the rotors/actuators. The coupling matrix Σ_R maps the torques/rotations of the motors into small translational lateral forces (then rotated into SR_i via R).

We know that the derivative of the rotation matrix R depends on the skew-symmetric matrix of the angular velocity:

$$\dot{R} = R \cdot S(\boldsymbol{\Omega}) \tag{1.4}$$

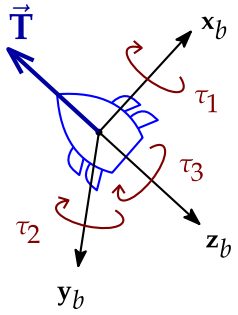
$S(\boldsymbol{\Omega})$ is important since it appears in the explicit expression of the term $J\dot{\boldsymbol{\Omega}}$ in Equation (1.2):

$$J\dot{\boldsymbol{\Omega}} = -S(\boldsymbol{\Omega})J\boldsymbol{\Omega} + \boldsymbol{\tau} + \boldsymbol{\tau}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + \boldsymbol{\tau}_g + \Sigma_T T\mathbf{z} \tag{1.5}$$

where:

- $\boldsymbol{\tau}_e$ are the torques induced by the external forces.
- $\Sigma_T T\mathbf{z}$ is a parasitic moment. Σ_T is a matrix that represents the geometric misalignment between the effective axis of the total thrust T and the center of mass of the vehicle. If the thrust vector does not pass exactly through the center of gravity, it generates a parasitic torque proportional to the thrust itself.
- $\boldsymbol{\tau}_g$ is the internal gyroscopic moment due to the inertia of the propellers rotating at high speed (particularly relevant in quadrotors when they change attitude rapidly).

We denote $\vec{\mathbf{T}} = -T\mathbf{z}_b$ the push in the opposite direction of $\mathbf{z}_b$ expressed in the vehicle frame SR_b.



A quadrotor is a UAV with four motors located at the extremities of a cross-shaped frame (Figure 1.4) and is controlled by imposing a desired angular speed on each rotor.

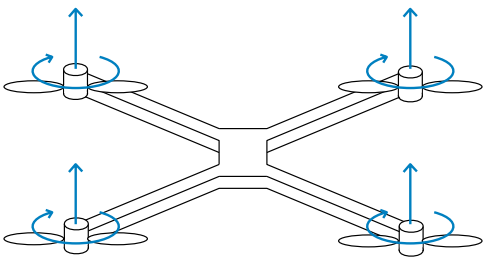


Figure 1.4: Schematic of a quadrotor.

Let's identify a disc on each rotor, which follows the profile of the propellers: we denote by f_i the scalar force on each propeller, which is perpendicular to the disc; its value is:

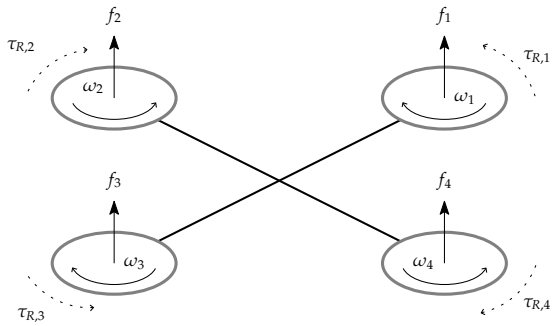
$$f_i = b\omega_i^2 \tag{1.6}$$

where ω_i is the angular speed of the i-th propeller, and b is a constant *thrust factor* that depends on the rotor geometry and profile, and on air density. The rotational speed of each blade is controlled by a low-level controller.

We denote $\tau_{R,i}$ the drag torque generated by the i-th rotor as it spins in the air:

$$\tau_{R,i} = d\omega_i^2 \tag{1.7}$$

where d is a *drag factor* that, like the thrust factor, depends on the rotor.



We denote φ, ϑ, ψ the roll-pitch-yaw angles of the vehicle with respect to the fixed reference frame SR_i, hence the already mentioned rotation matrix is:

$${}^iR_b = \begin{pmatrix} \cos\psi\cos\vartheta & \cos\psi\sin\vartheta\sin\varphi - \sin\psi\cos\varphi & \cos\psi\sin\vartheta\cos\varphi + \sin\psi\sin\varphi \\ \sin\psi\cos\vartheta & \sin\psi\sin\vartheta\sin\varphi + \cos\psi\cos\varphi & \sin\psi\sin\vartheta\cos\varphi - \sin\varphi\cos\psi \\ -\sin\vartheta & \cos\vartheta\sin\varphi & \cos\vartheta\cos\varphi \end{pmatrix}. \tag{1.8}$$

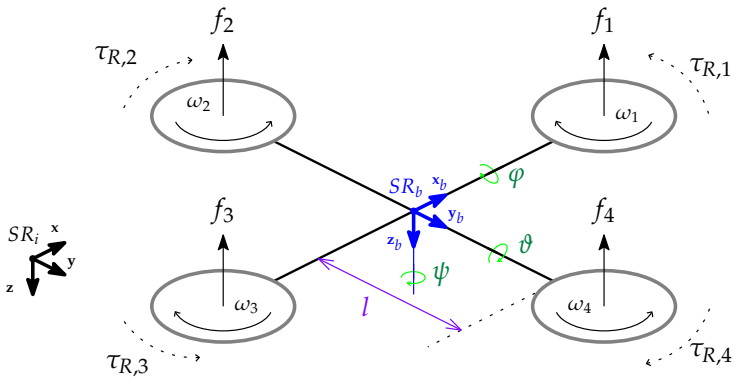


Figure 1.5: Configuration of the quadrotor.

The linear velocity $\mathbf{V}$ of our quadrotor is expressed in SR_i while the angular velocity $\mathbf{\Omega}$ is expressed in SR_b, we can make the reference system explicit using superscripts as follows:

$${}^i\mathbf{V} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}, \quad {}^b\mathbf{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix}. \tag{1.9}$$

Let $\omega_1, \omega_2, \omega_3, \omega_4$ be our control inputs (the velocity of the rotors), f_i and $\tau_{R,i}$ are derived from these quantities as in Equations (1.6) and (1.7). The total thrust will be:

$$T = f_1 + f_2 + f_3 + f_4. \tag{1.10}$$

We recall that on the quadrotor, a force $\vec{\mathbf{T}} = -T\mathbf{z}_b$ acts in the same direction as $\mathbf{z}_b$ but with the opposite sense. Intuitively, if the forces f_i are identical, we can expect the quadrotor to accelerate towards $-\mathbf{z}_b$ without rotating on itself. If the forces are not identical, they may cause a rotation of the body. The

torques generated on the axes $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ are the following:

$$\tau_\varphi = l(f_2 - f_4), \tag{1.11}$$
$$\tau_\vartheta = l(f_1 - f_3), \tag{1.12}$$
$$\tau_\psi = -\tau_{R,1} + \tau_{R,2} - \tau_{R,3} + \tau_{R,4}. \tag{1.13}$$

l represents the arm length (or lever arm) of the quadrotor, as shown in Figure 1.5.

For simplicity, we can assume that the inertia matrix shown in Equation (1.2) is diagonal:

$$J = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix} \tag{1.14}$$

the angular velocity instead can be expressed as a function of the roll-pitch-yaw angles:

$$\boldsymbol{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\sin\vartheta \\ 0 & \cos\varphi & \cos\vartheta\sin\varphi \\ 0 & -\sin\varphi & \cos\vartheta\cos\varphi \end{pmatrix} \begin{pmatrix} \dot{\varphi} \\ \dot{\vartheta} \\ \dot{\psi} \end{pmatrix}. \tag{1.15}$$

Let's see in detail what the applied forces and moments are. We write explicitly the sum of forces of the translational dynamics (that we recall is expressed in the fixed frame SR_i):

$$\sum_j \mathbf{F}_j = \underbrace{\begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix}}_{\text{weight}} + \underbrace{{}^iR_b \begin{pmatrix} 0 \\ 0 \\ -T \end{pmatrix}}_{\text{actuation}} + \mathbf{F}_A + \mathbf{F}_D \tag{1.16}$$

where $\mathbf{F}_A$ are the aerodynamics-related forces, which mainly include the aerodynamic resistance of the body and arms of the aircraft to advancement, as well as any parasitic lift effects generated by the shape of the drone or by the rotor discs in translation, while $\mathbf{F}_D$ are the disturbances (for example, wind).

The sum of moments is the following:

$$\sum_j \mathbf{M}_j = \underbrace{\begin{pmatrix} L_\varphi \\ L_\vartheta \\ L_\psi \end{pmatrix}}_{\text{gyroscopic effects}} + \underbrace{\begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}}_{\text{actuation}} + \boldsymbol{\tau}_A + \boldsymbol{\tau}_D. \tag{1.17}$$

$\boldsymbol{\tau}_A$ is the resulting torque induced by the distributed aerodynamic forces when they do not have an application point coinciding with the center of gravity of the vehicle, combined with the rotational drag that the air exerts against the rotations of the rigid body in roll, pitch and yaw. $\boldsymbol{\tau}_D$ is the moment produced by the disturbances.

Definition 1.1.1 : State of the Quadrotor

Given the previous premises, we define the state of a quadrotor as a vector in $\mathbb{R}^{12}$ defined as follows:

$$\boldsymbol{\xi} = (x, \quad y, \quad z, \quad v_x, \quad v_y, \quad v_z, \quad \varphi, \quad \vartheta, \quad \psi, \quad p, \quad q, \quad r)^\top \tag{1.18}$$

Here x, y, z are the coordinates of the position of the center of mass of the quadrotor expressed in SR_i, v_x, v_y, v_z its velocity, φ, ϑ, ψ the roll-pitch-yaw angles between SR_i and SR_b, and p, q, r the angular velocity of the quadrotor expressed in SR_b.

The evolution in time of the quadrotor state is given by a dynamical system of the following form:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \tag{1.19}$$

where $\mathbf{u}$ are the inputs of the system defined as follows:

$$\mathbf{u} = \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{1.20}$$

these quantities depend on f_i, $\tau_{R,i}$, which in turn depend on the angular velocity of the rotors ω_i, which we assume we can control perfectly.

Definition 1.1.2 : Time Evolution of the Quadrotor

Let Ω_r be the the signed residual speed of the blades

$$\Omega_r = -\omega_1 + \omega_2 - \omega_3 + \omega_4$$

and I_r the inertia of the blades; the explicit expression of the system in Equation (1.19) is the following:

$$\dot{x} = v_x \tag{1.21}$$
$$\dot{y} = v_y \tag{1.22}$$
$$\dot{z} = v_z \tag{1.23}$$
$$\dot{v}_x = F_{A,x} - (\cos(\psi)\sin(\vartheta)\cos(\varphi) + \sin(\psi)\sin(\varphi))\frac{T}{m} \tag{1.24}$$
$$\dot{v}_y = F_{A,y} - (\sin(\psi)\sin(\vartheta)\cos(\varphi) - \sin(\varphi)\cos(\psi))\frac{T}{m} \tag{1.25}$$
$$\dot{v}_z = F_{A,z} + g - \cos(\vartheta)\cos(\varphi)\frac{T}{m} \tag{1.26}$$
$$\dot{\varphi} = p + \sin(\varphi)\tan(\vartheta)q + \cos(\varphi)\tan(\vartheta)r \tag{1.27}$$
$$\dot{\vartheta} = \cos(\varphi)q - \sin(\varphi)r \tag{1.28}$$
$$\dot{\psi} = \sin(\varphi)\sec(\vartheta)q + \cos(\varphi)\sec(\vartheta)r \tag{1.29}$$
$$\dot{p} = \tau_{A,x} + \frac{I_r}{I_x}q\Omega_r + \frac{I_y - I_z}{I_x}qr + \frac{\tau_\varphi}{I_x} \tag{1.30}$$
$$\dot{q} = \tau_{A,y} - \frac{I_r}{I_y}p\Omega_r + \frac{I_z - I_x}{I_y}pr + \frac{\tau_\vartheta}{I_y} \tag{1.31}$$
$$\dot{r} = \tau_{A,z} + \frac{I_x - I_y}{I_z}pq + \frac{\tau_\psi}{I_z}. \tag{1.32}$$

1.2 Quadrotor Control

The model in Definition 1.1.2 is too complex for control purposes. We neglect aerodynamics and gyroscopic effects, and we assume small variations of φ and ϑ such that

$$\begin{pmatrix}\dot{\varphi} & \dot{\vartheta} & \dot{\psi}\end{pmatrix}^\top \simeq \begin{pmatrix}p & q & r\end{pmatrix}^\top \tag{1.33}$$

we assume that the shape is symmetric, and we neglect the disturbances.

Definition 1.2.1 : Simplified Time Evolution of the Quadrotor

$$\ddot{x} = -(\cos(\psi)\sin(\vartheta)\cos(\varphi) + \sin(\psi)\sin(\varphi))\frac{T}{m} \tag{1.34}$$

$$\ddot{y} = -(\sin(\psi)\sin(\vartheta)\cos(\varphi) - \sin(\varphi)\cos(\psi))\frac{T}{m} \tag{1.35}$$

$$\ddot{z} = -\cos(\vartheta)\cos(\varphi)\frac{T}{m} + g \tag{1.36}$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \tag{1.37}$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \tag{1.38}$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{1.39}$$

Note how we changed the notation for the acceleration ($\ddot{x} = \dot{v}_x$, $\ddot{y} = \dot{v}_y$, $\ddot{z} = \dot{v}_z$).

1.2.1 Linear Control

Let's denote

$$\mathbf{p} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad \mathbf{\Theta} = \begin{pmatrix} \varphi \\ \vartheta \\ \psi \end{pmatrix}, \quad \mathbf{\Gamma} = \begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{1.40}$$

we rewrite the state as

$$\boldsymbol{\xi} = \begin{pmatrix} \mathbf{p}^\top & \dot{\mathbf{p}}^\top & \mathbf{\Theta}^\top & \dot{\mathbf{\Theta}}^\top \end{pmatrix}^\top \tag{1.41}$$

with our input vector:

$$\mathbf{u} = \begin{pmatrix} T \\ \mathbf{\Gamma} \end{pmatrix}. \tag{1.42}$$

Let's separate the translational and rotational dynamics:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - {}^iR_b(\mathbf{\Theta})\frac{T}{m}\mathbf{e}_3 \tag{1.43}$$

$$\ddot{\mathbf{\Theta}} = J^{-1}\mathbf{\Gamma}. \tag{1.44}$$

$\mathbf{e}_3 = (0,\ 0,\ 1)^\top$ is the unit vector pointing downward, previously denoted $\mathbf{z}$. Managing the nonlinear dynamics is hard for the real-time controller. We apply linearization around an equilibrium point, in particular in a hovering condition. Let

$$\mathbf{p}_e = \begin{pmatrix} x_e \\ y_e \\ z_e \end{pmatrix}, \quad \mathbf{\Theta}_e = \begin{pmatrix} \varphi_e \\ \vartheta_e \\ \psi_e \end{pmatrix} \tag{1.45}$$

be our equilibrium point. We assume that our drone is perfectly horizontal, i.e., $\varphi_e = \vartheta_e = 0$ with also $\psi_e = 0$, and there are no active torques: $\mathbf{\Gamma} = \mathbf{0}$. The thrust perfectly balances the gravity:

$$T_e = mg. \tag{1.46}$$

The full *hovering equilibrium* condition is expressed as follows:

$$\mathbf{\Theta}_e = \mathbf{0}, \quad \mathbf{\Gamma} = \mathbf{0}, \quad T_e = mg, \quad \dot{\mathbf{p}}_e = \mathbf{0}, \quad \dot{\mathbf{\Theta}}_e = \mathbf{0}. \tag{1.47}$$

We denote $\ddot{\mathbf{p}} = \mathbf{F}(\mathbf{\Theta}, T)$ the translational part in Equation (1.43). We approximate a small variation of $\ddot{\mathbf{p}}$ with its first order expansion around the equilibrium point:

$$\delta\ddot{\mathbf{p}} \simeq \frac{\partial \mathbf{F}}{\partial \mathbf{\Theta}}\bigg|_e \delta\mathbf{\Theta} + \frac{\partial \mathbf{F}}{\partial T}\bigg|_e \delta T \tag{1.48}$$

with δT and $\delta\boldsymbol{\Theta}$ being small variations. To compute the expansion, we need to compute the variation of the rotation matrix ${}^{i}R_b(\boldsymbol{\Theta})$, given by the small variation $\delta\boldsymbol{\Theta}$ of the angles around $\boldsymbol{\Theta}_e = \mathbf{0}$. We have

$$\delta\,{}^{i}R_b = \frac{\partial {}^{i}R_b}{\partial\varphi}\Big|_{\mathbf{0}}\delta\varphi + \frac{\partial {}^{i}R_b}{\partial\vartheta}\Big|_{\mathbf{0}}\delta\vartheta + \frac{\partial {}^{i}R_b}{\partial\psi}\Big|_{\mathbf{0}}\delta\psi. \tag{1.49}$$

We recall that ${}^{i}R_b(\boldsymbol{\Theta}) = R_z(\psi)R_y(\vartheta)R_x(\varphi)$, hence (without showing the whole calculation):

$$\delta^{i}R_b = \begin{pmatrix} 0 & -\delta\psi & \delta\vartheta \\ \delta\psi & 0 & -\delta\varphi \\ -\delta\vartheta & \delta\varphi & 0 \end{pmatrix} \tag{1.50}$$

this is exactly the skew-symmetric matrix $S(\delta\boldsymbol{\Theta})$. Now we can make explicit the formula in Equation (1.48):

$$\frac{\partial \mathbf{F}}{\partial\boldsymbol{\Theta}}\Big|_e \delta\boldsymbol{\Theta} = -gS(\delta\boldsymbol{\Theta})\mathbf{e}_3 \tag{1.51}$$

$$\frac{\partial \mathbf{F}}{\partial T}\Big|_e \delta T = -\frac{\delta T}{m}\mathbf{e}_3 \tag{1.52}$$

$$\implies \delta\ddot{\mathbf{p}} \simeq -g\begin{pmatrix} \delta\vartheta \\ -\delta\varphi \\ 0 \end{pmatrix} - \frac{\delta T}{m}\mathbf{e}_3. \tag{1.53}$$

Now we drop the notation with the δ (small variations), the quadrotor system linearized around an equilibrium point is governed by the following equations:

$$\ddot{x} = -g\vartheta, \qquad \ddot{\vartheta} = \frac{\tau_\vartheta}{I_y}, \tag{1.54}$$

$$\ddot{y} = g\varphi, \qquad \ddot{\varphi} = \frac{\tau_\varphi}{I_x}, \tag{1.55}$$

$$\ddot{z} = -\frac{(T - mg)}{m}, \qquad \ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{1.56}$$

This system is linear and decoupled:

- In order to control x, we can control ϑ through τ_ϑ, hence x is controllable with a chain of four integrators.
- In order to control y, we can control φ through τ_φ, hence y is controllable with a chain of four integrators.
- In order to control z we use T in a chain of two integrators.
- In order to control ψ we use τ_ψ in a chain of two integrators.

1.2.2 Basic Control and Planning Problems

The basic control objectives for a quadrotor are the following:

- attitude control,
- height control,
- position control,
- hovering-to-hovering motion,
- waypoint navigation,
- trajectory planning and tracking (see Chapter 4 for the planning),
- obstacle avoidance.

With **attitude control**, we mean the process of determining the torques $\tau_\varphi, \tau_\vartheta, \tau_\psi$ needed to obtain a stable desired attitude $\varphi_d, \vartheta_d, \psi_d$. A simple solution is a proportional-derivative (PD) action on the error.

Definition 1.2.2 : PD Controller for the Attitude

$$\tau_\varphi = K_{\varphi p}(\varphi_d - \varphi) + K_{\varphi d}(\dot{\varphi}_d - \dot{\varphi}) \tag{1.57}$$
$$\tau_\vartheta = K_{\vartheta p}(\vartheta_d - \vartheta) + K_{\vartheta d}(\dot{\vartheta}_d - \dot{\vartheta}) \tag{1.58}$$
$$\tau_\psi = K_{\psi p}(\psi_d - \psi) + K_{\psi d}(\dot{\psi}_d - \dot{\psi}). \tag{1.59}$$

For the **height control** we consider the dynamical model, in particular Equation (1.36). We choose the value of T that linearizes the system.

Definition 1.2.3 : Height Control with FBL

To bring the quadrotor to a desired height z_d and keep it there, we apply feedback linearization (FBL) plus a PD controller.

$$T = \frac{m}{\cos\vartheta\cos\varphi}[g - \ddot{z}_d - K_{zp}(z_d - z) - K_{zd}(\dot{z}_d - \dot{z})]. \tag{1.60}$$

Observation 1.2.1

A quadrotor is an underactuated system: it has 6 degrees of freedom in space (x, y, z and three attitude angles), but only 4 independent control inputs (the total thrust T and the three torque moments $\tau_\varphi, \tau_\vartheta, \tau_\psi$). There are no side motors to push the drone directly along the x axis or the y axis. To translate horizontally, the drone must first tilt (rotate) so that a component of the thrust T points in the desired direction.

Given the underactuated nature of the quadrotor, while we are controlling the altitude z we cannot directly control x and y, hence we adopt a *hierarchical* control scheme.

1. First, we assume that we have direct control on x and y, with two **virtual control inputs** u_x and u_y:

$$\ddot{x} = u_x \tag{1.61}$$
$$\ddot{y} = u_y. \tag{1.62}$$

2. These virtual control inputs are defined as a PD action on the error for stabilization:

$$u_x = \ddot{x}_d + K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) \tag{1.63}$$
$$u_y = \ddot{y}_d + K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}). \tag{1.64}$$

3. From the dynamics in Equations (1.34) and (1.35), we denote T_x and T_y the normalized components of the thrust along the inertial axes x and y:

$$\ddot{x} = \underbrace{(-\cos\varphi\sin\vartheta\cos\psi - \sin\varphi\sin\psi)}_{T_x}\frac{T}{m} = T_x\frac{T}{m} \tag{1.65}$$
$$\ddot{y} = \underbrace{(-\cos\varphi\sin\vartheta\sin\psi + \sin\varphi\cos\psi)}_{T_y}\frac{T}{m} = T_y\frac{T}{m}. \tag{1.66}$$

4. Let's equate the real expressions to the virtual accelerations:

$$T_x \frac{T}{m} = u_x \implies T_x = \frac{m}{T} u_x = \frac{m}{T} \left[K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) + \ddot{x}_d \right] \tag{1.67}$$

$$T_y \frac{T}{m} = u_y \implies T_y = \frac{m}{T} u_y = \frac{m}{T} \left[K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}) + \ddot{y}_d \right] \tag{1.68}$$

   in this way we get the desired value for T_x and T_y in order to achieve u_x and u_y.

5. At this point we have to solve (1.65) and (1.66) for the roll and the pitch to get the desired values φ_d and ϑ_d:

$$\varphi_d = \arcsin \left(T_y \cos\psi - T_x \sin\psi \right) \tag{1.69}$$

$$\vartheta_d = -\arcsin \left(\frac{T_x \cos\psi + T_y \sin\psi}{\cos\varphi_d} \right). \tag{1.70}$$

6. Finally, the inner loop will generate the desired values for $(\tau_\varphi, \tau_\vartheta, \tau_\psi)$ to achieve φ_d and ϑ_d.

To recap the whole control scheme:

- The quadrotor has no laterally oriented propellers, and generates a single main thrust T perpendicular to its frame. Consequently, there is no independent direct force on x and y.
- T is set as a priority to support the weight of the quadrotor and govern the z altitude.
- If we keep the drone perfectly horizontal, the thrust only points upwards. If, however, we tilt it in roll (φ) and pitch (ϑ), the thrust vector is decomposed: one part continues to compensate for gravity along z, while the other is projected onto the horizontal plane.
- The position loop calculates how much horizontal acceleration $(\ddot{x}, \ddot{y})$ is needed to reach the target and, working backwards, determines exactly which roll and pitch angles (φ_d, ϑ_d) project the necessary amount of thrust T in that precise direction.

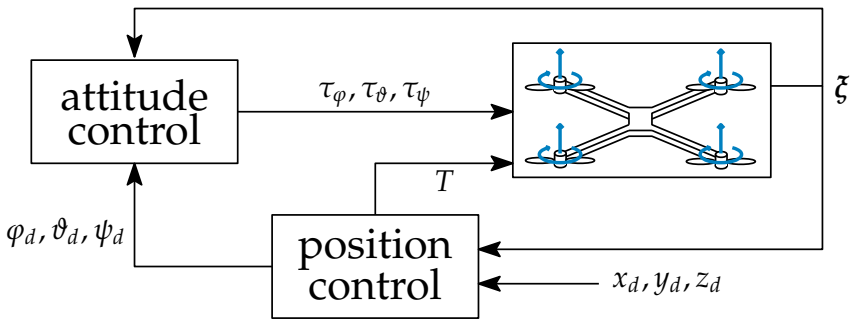


Figure 1.6: Structure of the control system.

CHAPTER

2

CONTROL METHODS

2.1 Backstepping

For a dynamical system:

$$\begin{cases} \dot{\mathbf{x}} = f(\mathbf{x}) \\ f(\mathbf{0}) = \mathbf{0} \end{cases} \tag{2.1}$$

we recall that a Lyapunov function is a scalar function that depends on $\mathbf{x}$, is continuously differentiable and positive definite:

$$V(\mathbf{x}) > 0, \quad \forall \mathbf{x} \neq \mathbf{0} \tag{2.2}$$

$$V(\mathbf{0}) = 0. \tag{2.3}$$

If $\dot{V} = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x})$ is negative definite, then the equilibrium $\mathbf{x} = \mathbf{0}$ is asymptotically stable. We also know that:

- If V is radially unbounded, the system is globally asymptotically stable.
- If $\exists c_1 > 0, c_2 > 0, c_3 > 0$ with $c_1 \leq c_2$ such that

  $$c_1 \|\mathbf{x}\|^2 \leq V \leq c_2 \|\mathbf{x}\|^2, \tag{2.4}$$

  $$\dot{V} \leq -c_3 \|\mathbf{x}\|^2 \tag{2.5}$$

  the system is globally exponentially stable.
- If $\dot{V}$ is negative semidefinite, the asymptotic stability can be proved using LaSalle's invariance principle.

In general, V is hard to find, so the backstepping method is used to construct both V and the control law; it renders an equilibrium point or a trajectory of a nonlinear system globally asymptotically stable (GAS).

The assumption is that our system is in strict-feedback (triangular) form; the quadrotor model satisfies this assumption. We will construct a simple two-dimensional example of how backstepping works. We have the following nonlinear system:

$$\dot{x}_1 = f_1(x_1) + g_1(x_1) x_2 \tag{2.6}$$

$$\dot{x}_2 = f_2(x_1, x_2) + g_2(x_1, x_2) u \tag{2.7}$$

with $x_1 = 0$ being an equilibrium for the first subsystem. Notice the triangular structure: the first subsystem that describes the dynamics of x_1 depends nonlinearly only on x_1, and linearly on x_2. The second subsystem depends linearly on u (control input) and nonlinearly on (x_1, x_2). We also assume that

$$\begin{matrix} g_1(x_1) \neq 0 \\ g_2(x_1, x_2) \neq 0 \end{matrix} \qquad \forall (x_1, x_2). \tag{2.8}$$

- **Step 1**: We stabilize x_1 by assuming that x_2 is a control input (can be chosen arbitrarily):

$$x_2 = \alpha(x_1) = \frac{1}{g_1(x_1)}[-f_1(x_1) - k_1 x_1] \tag{2.9}$$

with the Lyapunov function $V_1 = \frac{1}{2}x_1^2$. This leads to a simple differential equation:

$$\dot{x}_1 = -k_1 x_1 \tag{2.10}$$

and

$$\dot{V}_1 = -k_1 x_1^2 \tag{2.11}$$

hence the controller will globally stabilize x_1 to 0.

- **Step 2**: In reality, x_2 has its own dynamics and is not necessarily equal to our *virtual* control input $\alpha(x_1)$, hence we define an error:

$$z = x_2 - \alpha(x_1) \tag{2.12}$$

the dynamics of this error is

$$\dot{z} = \dot{x}_2 - \dot{\alpha} \tag{2.13}$$

we substitute $\dot{x}_2$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1}\dot{x}_1 \tag{2.14}$$

notice how we dropped the dependencies (for the sake of notational simplicity). Now we substitute $\dot{x}_1$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1}(f_1 + g_1 x_2). \tag{2.15}$$

For the system in Equation (2.15) we consider the following Lyapunov function:

$$V = \frac{1}{2}x_1^2 + \frac{1}{2}z^2$$

that is zero only at $x_1 = 0, z = 0$. We want to stabilize z to 0 (nullify the error on x_2). The derivative is:

$$\dot{V} = -k_1 x_1^2 + x_1 g_1 z + z(f_2 + g_2 u - \dot{\alpha}). \tag{2.16}$$

To reach the desired behavior, we choose u (our real control input) in order to make $\dot{V}$ negative definite:

$$u = \frac{1}{g_2}[-f_2 + \dot{\alpha} - k_2 z - x_1 g_1] \implies \tag{2.17}$$

$$\dot{V} = -k_1 x_1^2 - k_2 z^2 \leq -2\min(k_1, k_2)V \tag{2.18}$$

with k_1, k_2 being two positive real numbers. The term $-k_2 z$ drives x_2 to $\alpha(x_1)$. This control law achieves exponential stability for the system in x_1 and z, and global asymptotic stability in x_1 and x_2. The achieved closed loop system in the error coordinates is

$$\begin{pmatrix} \dot{x}_1 \\ \dot{z} \end{pmatrix} = \begin{pmatrix} -k_1 & g_1 \\ -g_1 & -k_2 \end{pmatrix} \begin{pmatrix} x_1 \\ z \end{pmatrix}. \tag{2.19}$$

The backstepping method can be recursively applied to a general triangular n-th order system:

$$\dot{x}_i = f_i(x_1, \ldots, x_i) + g_i(x_1, \ldots, x_i)x_{i+1}, \qquad i = 1, \ldots, n-1 \tag{2.20}$$

$$\dot{x}_n = f_n(\mathbf{x}) + g_n(\mathbf{x})u. \tag{2.21}$$

For each step we define the error z_i, with the addition of a term to the Lyapunov function:

$$\dot{z}_i = -k_i z_i - g_{i-1} z_{i-1} + g_i z_{i+1} \tag{2.22}$$
$$\dot{V} = -\sum_i k_i z_i^2. \tag{2.23}$$

The backstepping method is a design procedure, not a defined control law, and it does not need to cancel the nonlinearities of a system to achieve stability. The problem is that the complexity of this procedure grows exponentially since each step needs the analytical derivative of the previous virtual input.

2.1.1 Backstepping for Height Control

Let's see an application of backstepping. The height dynamics of the quadrotor is the following:

$$\dot{z} = v_z \tag{2.24}$$
$$\dot{v}_z = g - \cos\varphi \cos\vartheta \frac{T}{m} \tag{2.25}$$

we choose T in order to compensate for gravity and linearize the system:

$$T = \frac{m}{\cos\varphi\cos\vartheta}(g - \tilde{T}) \implies \dot{v}_z = \tilde{T} \tag{2.26}$$

with $\tilde{T}$ to be chosen later. Given the desired height $z_d(t)$ we define the error:

$$e_z = z_d - z \implies \dot{e}_z = \dot{z}_d - v_z. \tag{2.27}$$

If the velocity v_z were the control input, we would choose:

$$v_z = K_{zp} e_z + \dot{z}_d \implies \dot{e}_z = -K_{zp} e_z \tag{2.28}$$

but v_z has its own dynamics, hence we consider the formula in Equation (2.28) as a virtual control input denoted v_z^d:

$$v_z^d = K_{zp} e_z + \dot{z}_d \tag{2.29}$$

so we can define the error between the virtual input and the real velocity:

$$e_{v_z} = v_z^d - v_z \tag{2.30}$$

furthermore:

$$\dot{e}_z = -K_{zp} e_z + e_{v_z} \tag{2.31}$$
$$\dot{e}_{v_z} = K_{zp}\dot{e}_z + \ddot{z}_d - \tilde{T}. \tag{2.32}$$

The Lyapunov function becomes:

$$V = \frac{1}{2}(e_z^2 + e_{v_z}^2) \tag{2.33}$$
$$\dot{V} = -K_{zp} e_z^2 + e_{v_z}(e_z + K_{zp}\dot{e}_z + \ddot{z}_d - \tilde{T}) \tag{2.34}$$

now we choose $\tilde{T}$ to make $\dot{V}$ negative definite

$$\tilde{T} = \ddot{z}_d + e_z + K_{zp}\dot{e}_z + K_{zd} e_{v_z} \implies \tag{2.35}$$
$$\dot{V} = -K_{zp} e_z^2 - K_{zd} e_{v_z}^2 \tag{2.36}$$

by putting everything together, the final control law is:

$$T = \frac{m}{\cos\varphi\cos\vartheta}\left(g - \ddot{z}_d - (1 + K_{zp}K_{zd})e_z - (K_{zp} + K_{zd})\dot{e}_z\right). \tag{2.37}$$

2.1.2 PD Compared With Backstepping

What are the differences between the PD seen in Section 1.2.2 and the backstepping? They share a similar structure, but with a different choice of coordinates. The control laws are identical if we set the gain of the PD as follows:

$$K_p = 1 + K_{zp}K_{zd} \tag{2.38}$$

$$K_d = K_{zp} + K_{zd}. \tag{2.39}$$

The PD controller aims to nullify the position and velocity errors $(e_z, \dot{e}_z)$, while the backstepping law considers $e_{v_z} = \dot{e}_z + K_{zp}e_z$ instead of $\dot{e}_z$, hence the desired velocity already contains a term related to the position correction.

The gains have different roles:

- K_{zp} controls the decay rate of the position error, when the velocity error is zero.
- K_{zd} is the decay rate of the velocity error (the inner loop of the backstepping).
- In the PD controller, the natural frequency and the damping ratio of the error dynamics are given by

$$\omega_n = \sqrt{K_p} \tag{2.40}$$

$$\zeta = \frac{K_d}{2\sqrt{K_p}}. \tag{2.41}$$

With the same choices for the gains $K_p = K_{zp}, K_d = K_{zd}$ the closed-loop poles are (by setting $K_p = K_d = 2$):

- **backstepping**: $-2 \pm i$
- **PD**: $-1 \pm i$ (twice as slow).

A PD controller can be obtained from the backstepping procedure. The backstepping provides a hierarchical tuning procedure, with an explicit Lyapunov function.

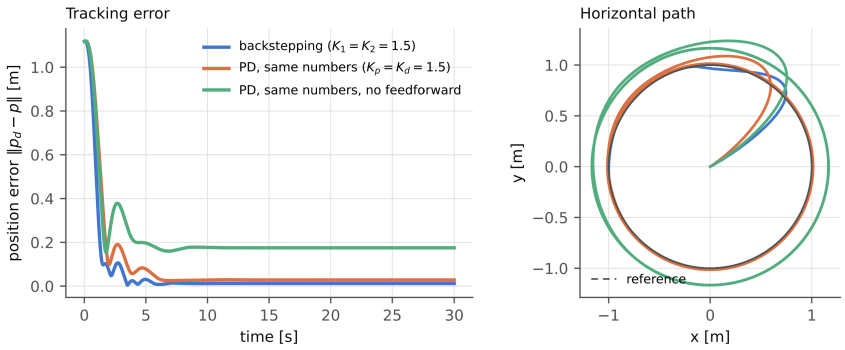


Figure 2.1: Simulated comparison between backstepping and PD.

2.1.3 Integral Backstepping

Now we consider the roll dynamics:

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} + h_\varphi \tag{2.42}$$

h_φ represents the nonlinear terms, for example the gyroscopic effects. We define the integral action of the error:

$$e_1 = \varphi_d - \varphi, \tag{2.43}$$

$$\chi = \int_0^t e_1 ds \tag{2.44}$$

then we define the desired (or virtual) roll velocity as:

$$\omega_d = c_1 e_1 + \dot{\varphi}_d + \lambda\chi \tag{2.45}$$

with c_1 and λ being control gains. We define a second error on the desired velocity:

$$e_2 = \omega_d - \dot{\varphi} \tag{2.46}$$

by deriving with respect to time the error equations we get:

$$\dot{e}_1 = -c_1 e_1 - \lambda\chi + e_2 \tag{2.47}$$
$$\dot{e}_2 = c_1 \dot{e}_1 + \ddot{\varphi}_d + \lambda e_1 - \ddot{\varphi} \tag{2.48}$$

to guarantee the asymptotic stability of the system we want to impose the following behavior for the error:

$$\dot{e}_2 = -c_2 e_2 - e_1 \tag{2.49}$$

hence we choose the following control law:

$$\tau_\varphi = I_x \left[(1 - c_1^2 + \lambda) e_1 + (c_1 + c_2) e_2 - c_1 \lambda \chi + \ddot{\varphi}_d - h_\varphi \right] \tag{2.50}$$

the same construction can be done for the altitude and the position. The stability of the system with this control law can be proved by defining the following Lyapunov candidate:

$$V = \frac{1}{2}\lambda\chi^2 + \frac{1}{2}e_1^2 + \frac{1}{2}e_2^2 \tag{2.51}$$
$$\dot{V} = -c_1 e_1^2 - c_2 e_2^2 \tag{2.52}$$

on the equilibrium $e_1 = e_2 = 0$ the system becomes:

$$\dot{e}_1 = -\lambda\chi \tag{2.53}$$

that is zero when $\chi = 0$ (GAS proved by LaSalle).

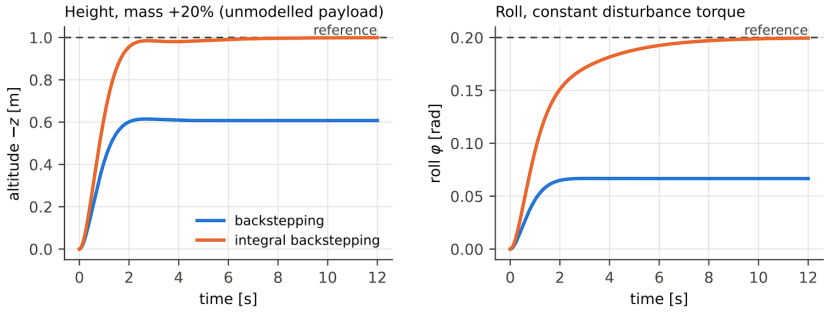


Figure 2.2: Performance comparison between backstepping and integral backstepping controllers, demonstrating that the integral action successfully eliminates steady-state errors under unmodeled payload (left) and constant disturbance torque (right).

2.2 Dynamic Feedback Linearization

One of the most important and effective ways to control a nonlinear dynamical system is to make it linear through feedback.

Definition 2.2.1 : Feedback Linearization

Feedback linearization is the exact cancellation of the nonlinearities by feedback and change of coordinates.

Recap of the model: For control purposes, we are using the (already seen) simplified model that assumes small changes for φ and ϑ

$$(\dot{\varphi}, \dot{\vartheta}, \dot{\psi}) \simeq (p, q, r)$$

and negligible aerodynamics/gyroscopic effects:

$$\ddot{x} = -(\cos\psi \sin\vartheta \cos\varphi + \sin\psi \sin\varphi)\frac{T}{m} \tag{2.54}$$

$$\ddot{y} = -(\sin\psi \sin\vartheta \cos\varphi - \sin\varphi \cos\psi)\frac{T}{m} \tag{2.55}$$

$$\ddot{z} = -\cos\vartheta \cos\varphi \frac{T}{m} + g \tag{2.56}$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \tag{2.57}$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \tag{2.58}$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{2.59}$$

We have used the symbol $\boldsymbol{\xi}$ for the state and $\mathbf{u}$ for the inputs. The compact form is

$$\begin{cases} \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\boldsymbol{\Theta}) \\ \ddot{\boldsymbol{\Theta}} = J^{-1}\boldsymbol{\Gamma}. \end{cases} \tag{2.60}$$

where we defined:

$$\mathbf{b}(\boldsymbol{\Theta}) = {}^iR_b(\boldsymbol{\Theta})\mathbf{e}_3 \tag{2.61}$$

$$\mathbf{p} = \begin{pmatrix} x & y & z \end{pmatrix}^\top \tag{2.62}$$

$$\boldsymbol{\Theta} = \begin{pmatrix} \varphi & \vartheta & \psi \end{pmatrix}^\top \tag{2.63}$$

$$\boldsymbol{\Gamma} = \begin{pmatrix} \tau_\varphi & \tau_\vartheta & \tau_\psi \end{pmatrix}^\top \tag{2.64}$$

$$J = \text{diag}(I_x, I_y, I_z). \tag{2.65}$$

2.2.1 Static Linearization

Before discussing the linearization of the quadrotor model, we will give some definitions for a general nonlinear system, but keep in mind the model in Equation (2.60), as we will need it later.

Consider the following nonlinear system:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \qquad \begin{matrix} \boldsymbol{\xi} \in \mathbb{R}^n \\ \mathbf{u} \in \mathbb{R}^m. \end{matrix} \tag{2.66}$$

If possible, we *want to find* a **static** state feedback and a change of coordinates:

$$\mathbf{u} = \alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v} \tag{2.67}$$

$$\mathbf{z} = \phi(\boldsymbol{\xi}) \tag{2.68}$$

with β a non-singular function and ϕ a diffeomorphism, such that the closed-loop system becomes linear and controllable:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \implies \tag{2.69}$$

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})(\alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v}) \tag{2.70}$$

since $\boldsymbol{\xi} = \phi^{-1}(\mathbf{z})$:

$$\frac{d}{dt}\phi^{-1}(\mathbf{z}) = f(\phi^{-1}(\mathbf{z})) + g(\phi^{-1}(\mathbf{z}))(\alpha(\phi^{-1}(\mathbf{z})) + \beta(\phi^{-1}(\mathbf{z}))\mathbf{v}) \implies \tag{2.71}$$

$$\dot{\mathbf{z}} = A\mathbf{z} + B\mathbf{v} \tag{2.72}$$

with A and B being constant matrices. This linearization is exact and holds whenever β and ϕ are well defined; the control design is completed by choosing $\mathbf{v}$ through the rules of linear control theory, and since ϕ is a diffeomorphism, by controlling $\mathbf{z}$ we can indirectly control $\boldsymbol{\xi}$.

2.2.2 Input-Output Linearization

With this method, we focus on a subset of the state vector that we want to control, by allowing (in general) a residual internal dynamics on the remaining variables. We choose m variables (same number of inputs) with a (sufficiently smooth) function h:

$$\boldsymbol{\eta} = h(\boldsymbol{\xi}) \in \mathbb{R}^m \tag{2.73}$$

We know that the inputs $\mathbf{u}$ are present in $\dot{\boldsymbol{\xi}}$, hence by differentiating $h(\boldsymbol{\xi})$, at some point one or more inputs of $\mathbf{u}$ will appear. $\boldsymbol{\eta}$ is a vector of functions η_i, $1 \leq i \leq m$, we have to differentiate each of these functions until *at least* one of the inputs of $\mathbf{u}$ appears.

Algorithm 1 Input-Output Linearization

```
for i = 1, ..., m do
    found ← false
    for r_i = 1, ..., +∞ do
        for j = 1, ..., m do
            if u_j is present in η_i^(r_i) then
                save η_i^(r_i)
                found ← true
                break
            end if
        end for
        if found then
            break
        end if
    end for
end for
```

The explicit expression of $\eta_i^{(r_i)}$ is

$$\eta_i^{(r_i)} = \underbrace{L_f^{r_i} h_i(\boldsymbol{\xi})}_{\text{Lie derivative}} + \sum_{j=1}^{m} L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi}) u_j. \tag{2.74}$$

where r_i is the relative degree of differentiation for the i-th component of $\boldsymbol{\eta}$. Clearly, h should be in the class C^r with $r = \max_i r_i$. The Lie derivatives are defined as follows:

$$L_f^0 h_i(\boldsymbol{\xi}) = h_i(\boldsymbol{\xi}) \tag{2.75}$$

$$L_f^1 h_i(\boldsymbol{\xi}) = L_f h_i(\boldsymbol{\xi}) = \frac{\partial h_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} f(\boldsymbol{\xi}) \tag{2.76}$$

$$L_f^k h_i(\boldsymbol{\xi}) = \frac{\partial (L_f^{k-1} h_i)}{\partial \boldsymbol{\xi}} f(\boldsymbol{\xi}). \tag{2.77}$$

$L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi})$ represents the Lie derivative of the intermediate scalar function $L_f^{r_i - 1} h_i(\boldsymbol{\xi})$ along the input vector field $g_j(\boldsymbol{\xi})$:

$$L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi}) = \frac{\partial (L_f^{r_i - 1} h_i(\boldsymbol{\xi}))}{\partial \boldsymbol{\xi}} g_j(\boldsymbol{\xi}). \tag{2.78}$$

About the Notation 2.2.1

When the superscript of a symbol is enclosed in round brackets, it denotes a derivative; if there are no brackets, it is an exponent or an index:

a^k is a raised to the power k (2.79)

$a^{(k)}$ is the k-th derivative of the function a. (2.80)

For the particular case of the Lie derivatives, the superscript k in L_f^k does not represent an exponent but an order of differentiation, even though it is not enclosed in brackets.

We can stack the m equations defined in (2.74) and write the system in compact form:

$$\begin{pmatrix} \eta_1^{(r_1)} \\ \vdots \\ \eta_m^{(r_m)} \end{pmatrix} = I(\boldsymbol{\xi}) + D(\boldsymbol{\xi})\mathbf{u} \tag{2.81}$$

$D(\boldsymbol{\xi})$ is called the **decoupling matrix**. If it is non-singular, then, for $\mathbf{v} \in \mathbb{R}^m$, choosing the following control law:

$$\mathbf{u} = D^{-1}(\boldsymbol{\xi})(-I(\boldsymbol{\xi}) + \mathbf{v}) \tag{2.82}$$

we achieve the following dynamic behavior:

$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m \tag{2.83}$$

we obtain m **decoupled chains of integrators** from v_i to η_i.

- if $\sum_i r_i = n$, we have **full state linearization**, and $\mathbf{z} = (\eta_i, \eta_i^{(1)}, \dots, \eta_i^{(r_i-1)})$, for $i = 1, \dots, m$, are coordinates.
- if $\sum_i r_i < n$, there will be an $(n - \sum_i r_i)$-dimensional internal dynamics, whose stability must be checked separately.

2.2.3 Linearization of the Quadrotor Model

From the quadrotor model in Equation (2.60) we consider as natural outputs the position and the yaw angle:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.84}$$

the model relative only to these variables is:

$$\ddot{\boldsymbol{\eta}} = \begin{pmatrix} \ddot{\mathbf{p}} \\ \ddot{\psi} \end{pmatrix} = \begin{pmatrix} g\mathbf{e}_3 \\ 0 \end{pmatrix} + \begin{pmatrix} -\frac{1}{m}\mathbf{b}(\boldsymbol{\Theta}) & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ 0 & 0 & 0 & \frac{1}{I_z} \end{pmatrix} \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}. \tag{2.85}$$

The quadrotor has six degrees of freedom and four control inputs; its underactuated nature makes it impossible to linearize the full state with a simple static feedback linearization $\mathbf{u} = \alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v}$. If we tried to linearize only the output statically, we would be left with non-linearized internal dynamics.

To overcome this problem, we must add a **dynamic compensator**, by introducing a set of ν additional states to the controller:

$$\boldsymbol{\zeta} \in \mathbb{R}^\nu \text{ (additional states)} \tag{2.86}$$

we have to specify a dynamics for these states:

$$\dot{\boldsymbol{\zeta}} = a(\boldsymbol{\xi}, \boldsymbol{\zeta}) + b(\boldsymbol{\xi}, \boldsymbol{\zeta})\mathbf{u} \tag{2.87}$$

defined by the functions a, b such that, together with the controller:

$$\mathbf{u} = c(\boldsymbol{\xi}, \boldsymbol{\zeta}) + d(\boldsymbol{\xi}, \boldsymbol{\zeta})\mathbf{v} \tag{2.88}$$

for some functions c, d the closed-loop system, of dimension $n + \nu$, is equivalent under a state transformation $\mathbf{z} = \phi(\boldsymbol{\xi}, \boldsymbol{\zeta})$ to a *linear controllable system*. Physically, in the quadrotor, the auxiliary state $\boldsymbol{\zeta}$ typically consists of integrators added on the thrust input T. By integrating the thrust twice, the true thrust becomes a state coordinate of the extended system, delaying the immediate appearance of the input into the derivatives and allowing the rotational dynamics to align with the translational ones without singularities.

Let's see the **algorithm of dynamic extension**: the construction procedure used to find the dynamic compensator and completely linearize the system.

1. We choose an m-dimensional output $\boldsymbol{\eta} = h(\boldsymbol{\xi})$.
2. We differentiate the outputs with respect to time until the inputs appear such that the decoupling matrix is non-singular (invertible).
3. When differentiating the components of $\boldsymbol{\eta}$, it often happens that some inputs appear at a lower differentiation order than others, making the decoupling matrix singular. We add integrators on those input channels: the inputs become states of the compensator, their derivatives become the new inputs.
4. If, at the end of the algorithm, the sum of the differentiation orders reached for each output equals the size of the extended state space:
$$\sum_{i=1}^{m} r_i = n + \nu$$
(where n is the size of the original physical state and ν is the total number of integrators added), *full input-state-output linearization* is obtained.
5. Applying the inversion law on the extended system, the behavior between the new inputs v_i and the physical outputs η_i is exactly equivalent to m independent and decoupled chains of pure integrators:
$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m. \tag{2.89}$$

The linearizing quadrotor output that we choose is the position together with the yaw angle:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.90}$$

The model is the one in Equation (2.85). The inputs appear in a non-singular way only after differentiating η_1, η_2, η_3 four times and η_4 twice. The explicit expression of the vector $\mathbf{b}$ in the model is

$$\mathbf{b}(\boldsymbol{\Theta}) = {}^iR_b(\boldsymbol{\Theta})\mathbf{e}_3 = \begin{pmatrix} \cos\psi \sin\vartheta \cos\varphi + \sin\psi \sin\varphi \\ \sin\psi \sin\vartheta \cos\varphi - \cos\psi \sin\varphi \\ \cos\vartheta \cos\varphi \end{pmatrix} \tag{2.91}$$

we will need its Jacobian, denoted B:

$$B = \frac{\partial \mathbf{b}}{\partial \boldsymbol{\Theta}} = \begin{pmatrix} \sin\psi \cos\varphi - \cos\psi \sin\vartheta \sin\varphi & \cos\psi \cos\vartheta \cos\varphi & \cos\psi \sin\varphi - \sin\psi \sin\vartheta \cos\varphi \\ -\cos\psi \cos\varphi - \sin\psi \sin\vartheta \sin\varphi & \sin\psi \cos\vartheta \cos\varphi & \sin\psi \sin\varphi + \cos\psi \sin\vartheta \cos\varphi \\ -\cos\vartheta \sin\varphi & -\sin\vartheta \cos\varphi & 0 \end{pmatrix}. \tag{2.92}$$

By differentiating η_1, η_2, η_3 twice we get the translational part of the model:

$$\begin{pmatrix} \ddot{\eta}_1 & \ddot{\eta}_2 & \ddot{\eta}_3 \end{pmatrix}^\top = \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\boldsymbol{\Theta}) \tag{2.93}$$

we keep differentiating until the inputs $T, \boldsymbol{\Gamma}$ appear:

$$\mathbf{p}^{(3)} = -\frac{1}{m}(\dot{T}\mathbf{b} + TB\dot{\boldsymbol{\Theta}}) \tag{2.94}$$

one last differentiation:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\boldsymbol{\Theta}} + T\dot{B}\dot{\boldsymbol{\Theta}} + TB\ddot{\boldsymbol{\Theta}}) \tag{2.95}$$

$\ddot{\boldsymbol{\Theta}}$ is directly mapped to $\boldsymbol{\Gamma}$ through the inertia matrix:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\boldsymbol{\Theta}} + T\dot{B}\dot{\boldsymbol{\Theta}} + TBJ^{-1}\boldsymbol{\Gamma}) \tag{2.96}$$

finally, the inputs $\boldsymbol{\Gamma}$ appear in the Cartesian snap $\mathbf{p}^{(4)}$. Since $\eta_4 = \psi$, we need just two differentiations in order to get a non-singular dependence on τ_ψ:

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{2.97}$$

In order to control $\mathbf{p}^{(4)}$ we have to choose $\ddot{T}$, hence this will be our input, while T and $\dot{T}$ become states:

$$\zeta_1 = T, \quad \zeta_2 = \dot{T} \tag{2.98}$$

the model becomes:

$$\begin{pmatrix} \mathbf{p}^{(4)} \\ \psi^{(2)} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{m}\left(2\zeta_2 B\,\dot{\boldsymbol{\Theta}} + \zeta_1 \dot{B}\,\dot{\boldsymbol{\Theta}}\right) \\ 0 \end{pmatrix}}_{I(\boldsymbol{\xi},\boldsymbol{\zeta})} + \underbrace{\begin{pmatrix} -\frac{1}{m}\mathbf{b} & -\frac{\zeta_1}{m} B J^{-1} \\ 0 & 0 \quad 0 \quad \frac{1}{I_z} \end{pmatrix}}_{D(\boldsymbol{\xi},\boldsymbol{\zeta})} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{2.99}$$

In order to have the inputs appear in a non-singular way, D must be non-singular. Its determinant is

$$\det D = -\frac{T^2 \cos\varphi}{m^3 I_x I_y I_z} \tag{2.100}$$

hence D is non-singular if and only if $T \neq 0$ ($T = 0$ corresponds to free fall) and $\varphi \neq \pm\frac{\pi}{2}$. Away from the singularities, the sum of the relative degrees is

$$\sum_i r_i = 4 + 4 + 4 + 2 = 14 = n + \nu \tag{2.101}$$

hence *full state linearization* with no zero dynamics is achieved. The new state $\mathbf{z}$ that we choose through a diffeomorphism ϕ is given by the outputs and their derivatives:

$$\mathbf{z} = \phi(\boldsymbol{\xi},\boldsymbol{\zeta}) = \begin{pmatrix} \mathbf{p} & \dot{\mathbf{p}} & \ddot{\mathbf{p}} & \dddot{\mathbf{p}} & \psi & \dot{\psi} \end{pmatrix}^\top \in \mathbb{R}^{14} \tag{2.102}$$

we can act on $\mathbf{p}^{(4)}$ hence its derivatives of lower orders become part of the state, with:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{\zeta_1}{m}\mathbf{b}(\boldsymbol{\Theta}) \tag{2.103}$$

$$\dddot{\mathbf{p}} = -\frac{1}{m}(\zeta_2 \mathbf{b}(\boldsymbol{\Theta}) + \zeta_1 B(\boldsymbol{\Theta})\dot{\boldsymbol{\Theta}}). \tag{2.104}$$

The linearizing and decoupling control law is

$$\mathbf{u} = D^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}). \tag{2.105}$$

the closed loop system becomes:

$$x^{(4)} = v_1 \tag{2.106}$$

$$y^{(4)} = v_2 \tag{2.107}$$

$$z^{(4)} = v_3 \tag{2.108}$$

$$\ddot{\psi} = v_4. \tag{2.109}$$

The system is linear in the coordinates $\mathbf{z}$. To track a trajectory $\boldsymbol{\eta}_d$ we define the error $e_i = \eta_{id} - \eta_i$ for each i and set a PD (with higher order derivatives):

$$v_i = \eta_{id}^{(4)} + \sum_{j=1}^{4} c_{i,j-1} e_i^{(j-1)} \qquad i = 1,2,3 \tag{2.110}$$

$$v_4 = \ddot{\eta}_{4d} + c_{4,0} e_4 + c_{4,1}\dot{e}_4. \tag{2.111}$$

The system is stable and the tracking error, with an appropriate choice of the gains $c_{i,j}$, converges exponentially. The reference $\boldsymbol{\eta}_d$ must be differentiable four times at least for the first three components (position) and at least two times for the yaw ψ. The dynamic compensator has the following time evolution:

$$\dot{\zeta}_1 = \zeta_2 \tag{2.112}$$

$$\dot{\zeta}_2 = D_1^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.113}$$

$$T = \zeta_1 \tag{2.114}$$

$$\tau_\varphi = D_2^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.115}$$

$$\tau_\vartheta = D_3^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.116}$$

$$\tau_\psi = D_4^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.117}$$

with D_i^{-1} being the i-th row of D^{-1}, hence $D_i^{-1}(\xi,\zeta)(-I(\xi,\zeta)+\mathbf{v})$ is the scalar result of a product between row and column.

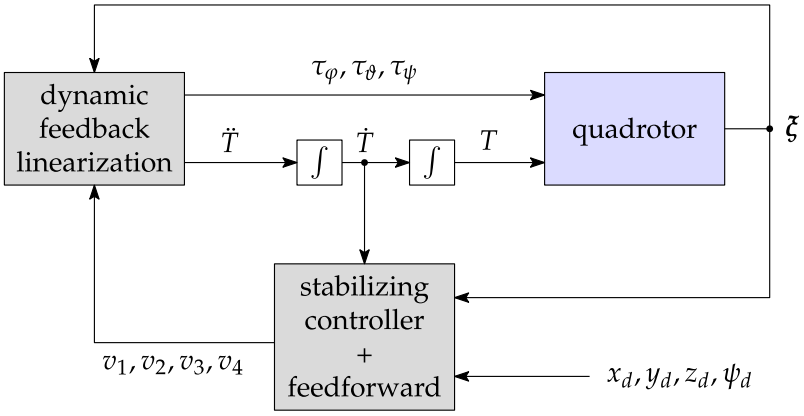


Figure 2.3: Block diagram of the control system.

2.2.4 Considering the Complete Model

In this section we discuss the dynamic feedback linearization for the quadrotor model, by removing the assumption of small angles. We neglect only the aerodynamics and gyroscopic effects:

$$\dot{x} = v_x \qquad \dot{\varphi} = p + s_\varphi \tan\vartheta q + c_\varphi \tan\vartheta r \tag{2.118}$$

$$\dot{y} = v_y \qquad \dot{\vartheta} = c_\varphi q - s_\varphi r \tag{2.119}$$

$$\dot{z} = v_z \qquad \dot{\psi} = s_\varphi \sec\vartheta q + c_\varphi \sec\vartheta r \tag{2.120}$$

$$\dot{v}_x = -(c_\psi s_\vartheta c_\varphi + s_\psi s_\varphi)\frac{T}{m} \qquad \dot{p} = \frac{I_y - I_z}{I_x} qr + \frac{\tau_\varphi}{I_x} \tag{2.121}$$

$$\dot{v}_y = -(s_\psi s_\vartheta c_\varphi - s_\varphi c_\psi)\frac{T}{m} \qquad \dot{q} = \frac{I_z - I_x}{I_y} pr + \frac{\tau_\vartheta}{I_y} \tag{2.122}$$

$$\dot{v}_z = g - c_\vartheta c_\varphi \frac{T}{m} \qquad \dot{r} = \frac{I_x - I_y}{I_z} pq + \frac{\tau_\psi}{I_z} \tag{2.123}$$

we are using the short notation for sin and cos. We denote $\mathbf{\Omega} = (p,q,r)^\top$ the angular velocity, the model in compact form can be rewritten as:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3 \tag{2.124}$$

$$\dot{R} = RS(\mathbf{\Omega}) \tag{2.125}$$

$$J\dot{\mathbf{\Omega}} = -\mathbf{\Omega} \times J\mathbf{\Omega} + \mathbf{\Gamma}. \tag{2.126}$$

We compute the dynamic feedback linearization (DFL) by setting the same output and extension:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.127}$$

$$\boldsymbol{\zeta} = \begin{pmatrix} T & \dot{T} \end{pmatrix}^\top \tag{2.128}$$

let's write out $\mathbf{p}^{(4)}$:

$$\mathbf{p}^{(3)} = \frac{d}{dt}\left(g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3\right) \tag{2.129}$$

$$= -\frac{T}{m}\frac{d}{dt}(R\mathbf{e}_3) - \frac{\dot{T}}{m} R\mathbf{e}_3 \tag{2.130}$$

$$= -\frac{1}{m} R(\dot{T}\mathbf{e}_3 + T\mathbf{\Omega} \times \mathbf{e}_3) \implies \tag{2.131}$$

$$\mathbf{p}^{(4)} = -\frac{1}{m} R \begin{pmatrix} T\dot{q} + 2\dot{T}q + Tpr \\ -T\dot{p} - 2\dot{T}p + Tqr \\ \ddot{T} - T(p^2+q^2) \end{pmatrix} \tag{2.132}$$

the snap $\mathbf{p}^{(4)}$ depends on $\dot{p}, \dot{q}, \ddot{T}$. We substitute $\dot{q}, \dot{p}$ as in Equations (2.121) and (2.122):

$$\mathbf{p}^{(4)} = -\frac{1}{m} R \begin{pmatrix} T(\frac{I_z - I_x}{I_y} pr + \frac{\tau_\vartheta}{I_y}) + 2\dot{T}q + Tpr \\ -T(\frac{I_y - I_z}{I_x} qr + \frac{\tau_\varphi}{I_x}) - 2\dot{T}p + Tqr \\ \ddot{T} - T(p^2 + q^2) \end{pmatrix} \tag{2.133}$$

In order to reorganize the snap equation, we define the following matrix and vector:

$$N = \begin{pmatrix} 0 & 0 & \frac{T}{I_y} \\ 0 & -\frac{T}{I_x} & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad \mathbf{n} = \begin{pmatrix} (1 + \frac{I_z - I_x}{I_y}) Tpr + 2\dot{T}q \\ (1 - \frac{I_y - I_z}{I_x}) Tqr - 2\dot{T}p \\ -T(p^2 + q^2) \end{pmatrix} \tag{2.134}$$

Equation (2.133) becomes:

$$\mathbf{p}^{(4)} = \underbrace{-\frac{1}{m} R\mathbf{n}}_{I_p(\xi,\zeta)} + \underbrace{-\frac{1}{m} RN}_{D_p(\xi,\zeta)} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \end{pmatrix} \tag{2.135}$$

I_p, D_p are the translational part of the matrices I, D of the model in the form in Equation (2.81). Notice how the snap $\mathbf{p}^{(4)}$ does not depend on τ_ψ. In the simplified model the yaw dynamics $\ddot{\psi}$ was a simple double integrator, as seen in Equation (2.59). This does not hold anymore, since:

$$\dot{\psi} = s_\varphi \sec\vartheta q + c_\varphi \sec\vartheta r \tag{2.136}$$

before deriving $\ddot{\psi}$ we introduce the following scalar:

$$I_\psi = \frac{s_\varphi \frac{I_z - I_x}{I_y} pr + c_\varphi \frac{I_x - I_y}{I_z} pq}{c_\vartheta} + \frac{(c_\varphi q - s_\varphi r)\dot{\varphi}}{c_\vartheta} + \frac{(s_\varphi q + c_\varphi r)\tan\vartheta\,\dot{\vartheta}}{c_\vartheta} \tag{2.137}$$

the yaw acceleration is exactly:

$$\ddot{\psi} = I_\psi + \frac{s_\varphi}{c_\vartheta I_y}\tau_\vartheta + \frac{c_\varphi}{c_\vartheta I_z}\tau_\psi \tag{2.138}$$

now both the control torques $\tau_\vartheta, \tau_\psi$ act on $\ddot{\psi}$. The complete model is:

$$\begin{pmatrix} \mathbf{p}^{(4)} \\ \ddot{\psi} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{m} R\mathbf{n} \\ I_\psi \end{pmatrix}}_{I(\xi,\zeta)} + \underbrace{\begin{pmatrix} & -\frac{1}{m} RN & & 0 \\ & & & 0 \\ & & & 0 \\ 0 & 0 & \frac{s_\varphi}{c_\vartheta I_y} & \frac{c_\varphi}{c_\vartheta I_z} \end{pmatrix}}_{D(\xi,\zeta)} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{2.139}$$

with

$$\det D = -\frac{T^2 \cos\varphi}{m^3 I_x I_y I_z \cos\vartheta} \tag{2.140}$$

hence the model has a singularity on $T = 0$ and $\varphi = \pm\frac{\pi}{2}$. Now we want to linearize this model, our virtual input is:

$$\mathbf{v} = \begin{pmatrix} v_1 & v_2 & v_3 & v_4 \end{pmatrix}^\top \tag{2.141}$$

with v_1, v_2, v_3 being relative to the snap and v_4 to the yaw. In order to linearize the snap we define the following vector:

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{pmatrix} = -mR^\top \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} - \mathbf{n} \tag{2.142}$$

The linearizing control law is

$$\ddot{T} = \mu_3 \tag{2.143}$$

$$\tau_\varphi = -\frac{I_x}{T}\mu_2 \tag{2.144}$$

$$\tau_\vartheta = \frac{I_y}{T}\mu_1 \tag{2.145}$$

$$\tau_\psi = \frac{I_z c_\vartheta}{c_\varphi}(v_4 - I_\psi) - \frac{I_z}{I_y}\tan\varphi\tau_\vartheta \tag{2.146}$$

the system becomes:

$$x^{(4)} = v_1 \tag{2.147}$$
$$y^{(4)} = v_2 \tag{2.148}$$
$$z^{(4)} = v_3 \tag{2.149}$$
$$\ddot{\psi} = v_4. \tag{2.150}$$

The following table compares the simplified model with the complete one:

	approximate	**complete**
rotational kinematics	$\dot{\boldsymbol{\Theta}} \simeq \boldsymbol{\Omega}$	$\dot{\boldsymbol{\Theta}} = M(\boldsymbol{\Theta})^{-1}\boldsymbol{\Omega}$
rotational dynamics	$\ddot{\boldsymbol{\Theta}} = J^{-1}\boldsymbol{\Gamma}$	$J\dot{\boldsymbol{\Omega}} = -\boldsymbol{\Omega} \times J\boldsymbol{\Omega} + \boldsymbol{\Gamma}$
jerk $\mathbf{p}^{(3)}$	$-\frac{1}{m}(\dot{T}\mathbf{b} + TB\dot{\boldsymbol{\Theta}})$	$-\frac{1}{m}R(Tq, -Tp, \dot{T})^\top$
position block of D	$-\frac{1}{m}(\mathbf{b} \quad TBJ^{-1})$	$-\frac{1}{m}RN(T)$
yaw row of D	$(0, 0, 0, \frac{1}{I_z})$	$\left(0, 0, \frac{s_\varphi}{c_\vartheta I_y}, \frac{c_\varphi}{c_\vartheta I_z}\right)$
det D	$-\dfrac{T^2 c_\varphi}{m^3 I_x I_y I_z}$	$-\dfrac{T^2 c_\varphi}{m^3 I_x I_y I_z c_\vartheta}$

2.2.5 Flatness

Definition 2.2.2 : Differentially Independent

Let $\mathbf{y}$ be a vector of quantities that changes over time. The components of $\mathbf{y}$ are **differentially independent** if there is no differential equation that links the individual components of $\mathbf{y}$ together, i.e. it is not possible to find an equation of the form:

$$\Phi(\mathbf{y}, \dot{\mathbf{y}}, \ddot{\mathbf{y}}, \dots) = \mathbf{0}. \tag{2.151}$$

Definition 2.2.3 : Differentially Flat System

A system $\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}, \mathbf{u})$ with $\mathbf{u} \in \mathbb{R}^m$ is **differentially flat** if there exist m outputs:

$$\mathbf{y} = h(\boldsymbol{\xi}, \mathbf{u}, \dot{\mathbf{u}}, \dots, \mathbf{u}^{(k)}) \tag{2.152}$$

called **flat outputs** such that the state $\boldsymbol{\xi}$ and the inputs $\mathbf{u}$ can be rewritten as functions of $\mathbf{y}$ and its derivatives (up to a finite order):

$$\boldsymbol{\xi} = \Psi_\xi(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(r_\xi)}) \tag{2.153}$$
$$\mathbf{u} = \Psi_u(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(r_u)}) \tag{2.154}$$

and the components of $\mathbf{y}$ are differentially independent.

If a system is differentially flat, the trajectories of the system are in one-to-one correspondence with arbitrary smooth curves $\mathbf{y}(t)$: there are no differential equations to integrate.

Observation 2.2.1

There does not exist a general test to check if a system is differentially flat, flat outputs are found by inspection, often with a physical meaning.

The quadrotor model is one of the best-known examples of a system with flat outputs, which are exactly:

$$\mathbf{y} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.155}$$

by considering the inputs

$$\mathbf{u} = \begin{pmatrix} T & \tau_\varphi & \tau_\vartheta & \tau_\psi \end{pmatrix}^\top \tag{2.156}$$

we want to derive the map Ψ such that:

$$\mathbf{u} = \Psi(\mathbf{y}, \dot{\mathbf{y}}, \ldots, \mathbf{y}^{(r)}) \tag{2.157}$$

first, we express T in function of $\ddot{\mathbf{p}}$, by the dynamic model:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3 \implies \tag{2.158}$$

$$T = m\|g\mathbf{e}_3 - \ddot{\mathbf{p}}\| \tag{2.159}$$

the versor of the thrust direction is:

$$\mathbf{b} = \frac{g\mathbf{e}_3 - \ddot{\mathbf{p}}}{\|g\mathbf{e}_3 - \ddot{\mathbf{p}}\|} \tag{2.160}$$

from the jerk equation we can define a new vector:

$$\mathbf{w} = -mR^\top \mathbf{p}^{(3)} \tag{2.161}$$

and get the following identities:

$$q = \frac{w_1}{T} \tag{2.162}$$

$$p = -\frac{w_2}{T} \tag{2.163}$$

$$\dot{T} = w_3 \tag{2.164}$$

and

$$r = \frac{c_\vartheta \dot{\psi} - s_\varphi q}{c_\varphi} \tag{2.165}$$

from the snap equation, we define:

$$\boldsymbol{\mu} = -mR^\top \mathbf{p}^{(4)} - \mathbf{n} \tag{2.166}$$

and we can write:

$$\ddot{T} = \mu_3 \tag{2.167}$$

$$\tau_\vartheta = \frac{I_y}{T}\mu_1 \tag{2.168}$$

$$\tau_\varphi = -\frac{I_x}{T}\mu_2 \tag{2.169}$$

$$\tau_\psi = \frac{I_z c_\vartheta}{c_\varphi}(\ddot{\psi} - I_\psi) - \frac{I_z}{I_y}\tan\varphi\tau_\vartheta \tag{2.170}$$

now that we have defined the map $\mathbf{u} = \Psi(\mathbf{y}, \dot{\mathbf{y}}, \ldots, \mathbf{y}^{(4)})$, we can define any trajectory at least four times differentiable $\mathbf{y}(t)$ and get immediately through Ψ the required inputs to execute $\mathbf{y}(t)$. The explicit expressions of the roll and pitch angles and of their derivatives in terms of the flat outputs, and their use for motion planning, are given in Chapter 4.

2.3 Geometric Control on $SO(3)$

The state space of the rotational dynamics is $\mathbb{R}^3 \times SO(3)$, this is a **topological obstruction**: due to this, it is impossible with a continuous feedback law to make an equilibrium point *globally asymptotically stable*. The best thing that we can do is to use the information about the geometry of the state space and define an error on that. By doing so, the best achievable result with a continuous feedback law is **almost global convergence** (a desired equilibrium attracts trajectories starting from almost all initial conditions, except a zero-measure set). The **geometric controller** is designed by using directly $SO(3)$ to represent the desired attitude of the UAV.

We recall the quadrotor dynamics:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3 \tag{2.171}$$

$$\dot{R} = RS(\boldsymbol{\Omega}) \tag{2.172}$$

$$J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} = \boldsymbol{\Gamma} \tag{2.173}$$

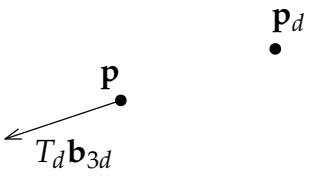
with $\boldsymbol{\Gamma}, T$ being the control inputs. We define a new operator $^\vee$: this is the inverse of the operator that assigns to a vector its associated skew-symmetric matrix:

$$A = S(\mathbf{a}) \tag{2.174}$$

$$A^\vee = (S(\mathbf{a}))^\vee = \mathbf{a} \tag{2.175}$$

The controller is divided in two loops, the outer loop for the position, the inner loop for the attitude. We denote $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ the unit vector columns of R (these vectors are the one denoted $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ in Figure 1.3). To impose a desired attitude, we define a desired rotation matrix R_d, with column vectors $\mathbf{b}_{1d}, \mathbf{b}_{2d}, \mathbf{b}_{3d}$. The two loops work as follows:

- The trajectory tracking controller takes in input a desired Cartesian position $\mathbf{p}_d$, and produces in output the required thrust intensity T and thrust direction $\mathbf{b}_{3d}$.



- $\mathbf{b}_{3d}$, and $\mathbf{b}_{1d}$ (desired yaw) are used to define R_d, that will be used by the attitude tracking controller to compute the required torques $\boldsymbol{\Gamma}$.
- $\boldsymbol{\Gamma}$ and T are the computed control inputs for the quadrotor dynamics.

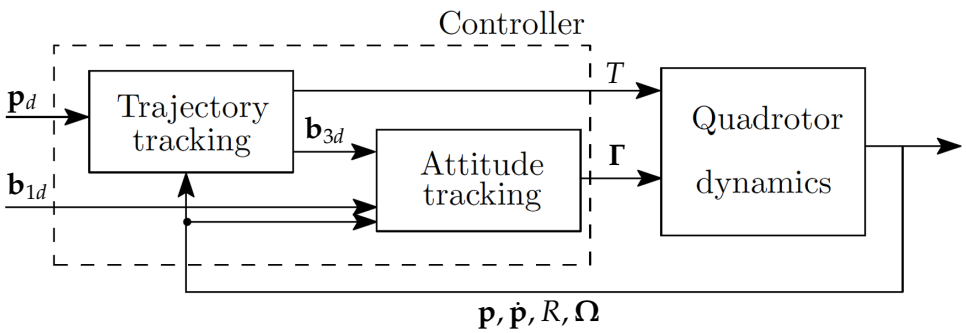


Figure 2.4: Block diagram of the geometric controller.

First of all we define the Cartesian error for the position:

$$\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d \tag{2.176}$$
$$\mathbf{e}_v = \dot{\mathbf{p}} - \dot{\mathbf{p}}_d \tag{2.177}$$

the resulting positional control action $\mathbf{a}$ is built with a PD law + gravity compensation + feedforward:

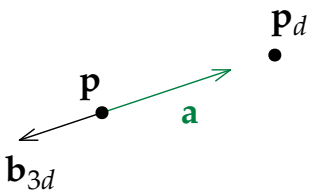
$$\mathbf{a} = -k_p\mathbf{e}_p - k_d\mathbf{e}_v - mg\mathbf{e}_3 + m\ddot{\mathbf{p}}_d. \tag{2.178}$$

The thrust force that we can impose is $-TR\mathbf{e}_3$, and we would like to make this equal to the positional control action:

$$-TR\mathbf{e}_3 = \mathbf{a} \tag{2.179}$$

we cannot assign directly $-TR\mathbf{e}_3$, but we can assign T and the control torques to control R. The thrust direction desired is the following:

$$\mathbf{b}_{3d} = -\frac{\mathbf{a}}{\|\mathbf{a}\|}. \tag{2.180}$$



This requires $\mathbf{a} \neq \mathbf{0}$ (no free fall). The thrust intensity is:

$$T = \|\mathbf{a}\|\mathbf{b}_{3d}^\top\mathbf{b}_3 \tag{2.181}$$

so T is the thrust intensity to actuate, now we have to describe how the inner attitude controller will achieve the desired attitude R_d defined by the desired thrust direction $\mathbf{b}_{3d}$. The attitude is constructed as follows:

- Let $\mathbf{b}_{1d}$ be the desired yaw. We build the vector:

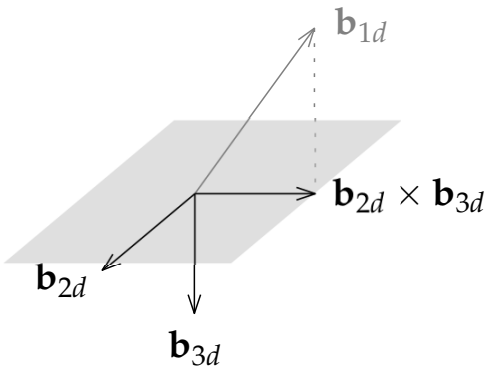
$$\mathbf{b}_{2d} = \frac{\mathbf{b}_{3d} \times \mathbf{b}_{1d}}{\|\mathbf{b}_{3d} \times \mathbf{b}_{1d}\|} \tag{2.182}$$

- $\mathbf{b}_{2d}$ and $\mathbf{b}_{3d}$ will be completed to an orthonormal basis by their cross product:

$$\mathbf{b}_{2d} \times \mathbf{b}_{3d} \tag{2.183}$$

  by this construction, $\mathbf{b}_{2d} \times \mathbf{b}_{3d}$ will be the projection of $\mathbf{b}_{1d}$ on the plane orthogonal to $\mathbf{b}_{3d}$. Ultimately:

$$R_d = \begin{pmatrix} \mathbf{b}_{2d} \times \mathbf{b}_{3d} & \mathbf{b}_{2d} & \mathbf{b}_{3d} \end{pmatrix}. \tag{2.184}$$



This construction is well defined if $\mathbf{b}_{1d}$ is not parallel to $\mathbf{b}_{3d}$. Now we have to define an error between matrices in $SO(3)$, so we define the error function $\Psi : SO(3) \times SO(3) \to [0, 2]$ as follows:

$$\Psi(R, R_d) = \frac{1}{2}\text{tr}\left[I - R_d^\top R\right] \tag{2.185}$$

this function has some important properties:

- $R_d^\top R$ is the relative rotation between the current and desired attitude.
- Let $R_d^\top R$ be the rotation about $\mathbf{r}$ of θ radians (axis-angle representation). Then:

$$\text{tr}\left[R_d^\top R\right] = 1 + 2\cos\theta \tag{2.186}$$

  hence:

$$\Psi(R, R_d) = 1 - \cos\theta \in [0, 2]. \tag{2.187}$$

- The error is zero if the matrices are identical:

$$\Psi(R, R_d) = 0 \iff R = R_d. \tag{2.188}$$

- The error is less than 1 if and only if the angle θ is less than $\frac{\pi}{2}$:

$$\Psi(R, R_d) < 1 \iff \theta < \frac{\pi}{2}. \tag{2.189}$$

- The error is maximum when the angle θ is exactly π:

$$\Psi(R, R_d) = 2 \iff \theta = \pi. \tag{2.190}$$

Proposition 2.3.1 : Almost Global Convergence

At the beginning of this section we have said that the geometric controller achieves almost global convergence. This means that asymptotic convergence is **guaranteed** if the error defined by Ψ between the current attitude R and desired attitude R_d is less than 2.

At each attitude R, asymptotic convergence is guaranteed on the subset:

$$L_2 = \{R_d \in SO(3) \text{ such that } \Psi(R, R_d) < 2\} \subset SO(3). \tag{2.191}$$

L_2 almost covers $SO(3)$, meaning that the set $SO(3) \setminus L_2$ has zero measure.

Now that we defined a desired attitude R_d and an error $\Psi(R, R_d)$ that is a measure of how distant we are from the desired configuration, we have to define how we choose the control torques $\mathbf{\Gamma}$. First of all we consider a *small perturbation* on R, let $\boldsymbol{\eta}$ be a vector of infinitesimal rotation, the perturbation is:

$$\delta R = RS(\boldsymbol{\eta}) \tag{2.192}$$

now we compute how Ψ changes when R changes (and R_d is constant). Since the trace operator is linear, the variation is pushed inside the operator:

$$\delta\Psi = \delta\frac{1}{2}\text{tr}\left[I - R_d^\top R\right] \tag{2.193}$$

$$= \frac{1}{2}\text{tr}\left[\delta I - \delta(R_d^\top R)\right] \tag{2.194}$$

$$= \frac{1}{2}\text{tr}\left[\delta I - (R_d^\top \delta R)\right] \tag{2.195}$$

I is constant hence δI is the zero matrix, then:

$$\delta\Psi = -\frac{1}{2}\text{tr}\left[R_d^\top RS(\boldsymbol{\eta})\right] \tag{2.196}$$

from a series of algebraic steps (that we omit), the variation can be rewritten as follows:

$$\delta\Psi = \underbrace{\frac{1}{2}\cdot\left(R_d^\top R - R^\top R_d\right)^\vee}_{\mathbf{e}_R}\cdot\boldsymbol{\eta} \tag{2.197}$$

$$= \mathbf{e}_R^\top\boldsymbol{\eta} \tag{2.198}$$

we have found the vector $\mathbf{e}_R$ that is the **gradient** of Ψ, represents the direction (expressed in the body frame) along which the error function Ψ grows most rapidly. It can be proved that, if $\mathbf{r}, \theta$ is the axis-angle representation of the relative rotation $R_d^\top R$, then the gradient can be written as:

$$\mathbf{e}_R = \sin\theta\mathbf{r} \tag{2.199}$$

furthermore its squared magnitude is:

$$\|\mathbf{e}_R\|^2 = \underbrace{\sin^2\theta\mathbf{r}^\top\mathbf{r} = \sin^2\theta}_{\mathbf{r}\text{ is a unit vector}} = \Psi(2-\Psi). \tag{2.200}$$

Let $\psi_2 < 2$ be a constant value decided a priori by design, it holds that, for any attitude R such that $\Psi(R, R_d) \leq \psi_2$, the following bounds hold:

$$\frac{1}{2}\|\mathbf{e}_R\|^2 \leq \Psi \leq \frac{1}{2-\psi_2}\|\mathbf{e}_R\|^2. \tag{2.201}$$

- From the lower bound ($\frac{1}{2}\|\mathbf{e}_R\|^2 \leq \Psi$) we deduce that if the configuration error Ψ tends to zero, then the error vector $\mathbf{e}_R$ must also tend to zero ($\Psi \to 0 \implies \|\mathbf{e}_R\| \to 0$).
- From the upper bound ($\Psi \leq \frac{1}{2-\psi_2}\|\mathbf{e}_R\|^2$) we deduce the vice versa: within the safe region $\Psi \leq \psi_2 < 2$, if the error vector $\mathbf{e}_R$ vanishes, then Ψ must also vanish, i.e. $R = R_d$.

Observation 2.3.1

The bounds on Equation (2.201) show that Ψ is bounded between two quadratic forms of the attitude error vector $\mathbf{e}_R$; therefore, within this sublevel set, $\Psi(R, R_d)$ is positive definite with respect to $\mathbf{e}_R$ (i.e., $\Psi = 0 \iff \mathbf{e}_R = \mathbf{0}$).

After defining the attitude error, we define the error on angular velocity; let $\boldsymbol{\Omega}_d = (p_d, q_d, r_d)$ be the desired angular velocity, the error $\mathbf{e}_\Omega$ is:

$$\mathbf{e}_\Omega = \boldsymbol{\Omega} - R^\top R_d\boldsymbol{\Omega}_d. \tag{2.202}$$

$\boldsymbol{\Omega}$ is expressed in the current body frame defined by R while $\boldsymbol{\Omega}_d$ is expressed in the desired body frame defined by R_d, hence $R_d\boldsymbol{\Omega}_d$ is the desired angular velocity in the base frame (also called inertial frame). $\mathbf{e}_\Omega$ is exactly the angular velocity vector that governs the time derivative of the relative rotation matrix $R_d^\top R$:

$$\frac{d}{dt}(R_d^\top R) = \dot{R}_d^\top R + R_d^\top\dot{R} \tag{2.203}$$

$$= -S(\boldsymbol{\Omega}_d)R_d^\top R + R_d^\top RS(\boldsymbol{\Omega}) \tag{2.204}$$

$$= R_d^\top R(S(\boldsymbol{\Omega}) - R^\top R_d S(\boldsymbol{\Omega}_d)R_d^\top R) \tag{2.205}$$

$$= R_d^\top R\, S(\boldsymbol{\Omega} - R^\top R_d\boldsymbol{\Omega}_d) \tag{2.206}$$

$$= R_d^\top R\, S(\mathbf{e}_\Omega). \tag{2.207}$$

It can be proved that the derivative of the attitude error function is the dot product between the gradient and the angular velocity error:

$$\dot{\Psi} = -\frac{1}{2}\mathrm{tr}\left[\frac{d}{dt}(R_d^\top R)\right] = -\frac{1}{2}\mathrm{tr}\left[R_d^\top RS(\mathbf{e}_\Omega)\right] = \frac{1}{2}\mathbf{e}_\Omega^\top(R_d^\top R - R^\top R_d)^\vee = \mathbf{e}_R^\top\mathbf{e}_\Omega. \tag{2.208}$$

Let's recap what we have done until now:

1. We defined the inner-outer loop structure of the geometric controller.
2. We have seen how the outer loop (trajectory) generates the desired thrust direction for the inner loop, defined by $\mathbf{b}_{3d}$.
3. The inner loop constructs the matrix R_d, and defines an error Ψ between the current attitude R and the desired one R_d.
4. We have defined the errors $\mathbf{e}_R = \nabla\Psi$ and $\mathbf{e}_\Omega$, and we have seen how $\dot{\Psi} = \mathbf{e}_R^\top \mathbf{e}_\Omega$.

What we have to do now?: We derive the dynamics of the errors, and choose the control torques $\boldsymbol{\Gamma}$ in order to achieve a closed loop system that asymptotically nullifies the errors.

The dynamics of the attitude error is the following:

$$\dot{\mathbf{e}}_R = \frac{1}{2}\left(R_d^\top R S(\mathbf{e}_\Omega) + S(\mathbf{e}_\Omega) R^\top R_d\right)^\vee \tag{2.209}$$

$$= \underbrace{\frac{1}{2}\left(\text{tr}\left[R^\top R_d\right] I - R^\top R_d\right)}_{C(R_d^\top R)} \mathbf{e}_\Omega \tag{2.210}$$

The matrix $\frac{1}{2}\left(\text{tr}\left[R^\top R_d\right] I - R^\top R_d\right)$ can be rewritten as $C(R_d^\top R)$ with C being a matrix with norm less or equal than 1:

$$\dot{\mathbf{e}}_R = C(R_d^\top R)\mathbf{e}_\Omega, \qquad \|C\| \leq 1. \tag{2.211}$$

So the velocity of the attitude error depends on the angular velocity error, its dynamics is:

$$\dot{\mathbf{e}}_\Omega = \dot{\boldsymbol{\Omega}} - \frac{d}{dt}(R^\top R_d \boldsymbol{\Omega}_d) \tag{2.212}$$

$$= \dot{\boldsymbol{\Omega}} + S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \underbrace{S(\boldsymbol{\Omega}_d)\boldsymbol{\Omega}_d}_{\mathbf{0}} - R^\top R_d \dot{\boldsymbol{\Omega}}_d \tag{2.213}$$

$$= \dot{\boldsymbol{\Omega}} + S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d \tag{2.214}$$

Let's multiply both sides by the quadrotor inertia matrix J:

$$J\dot{\mathbf{e}}_\Omega = J\dot{\boldsymbol{\Omega}} + J\left(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d\right) \tag{2.215}$$

we substitute $J\dot{\boldsymbol{\Omega}}$ with the dynamics in Equation (2.173):

$$J\dot{\mathbf{e}}_\Omega = \boldsymbol{\Gamma} - \boldsymbol{\Omega} \times J\boldsymbol{\Omega} + J\left(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d\right). \tag{2.216}$$

The control input that we choose to stabilize the system is the following:

$$\boldsymbol{\Gamma} = -k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} - J(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d) \tag{2.217}$$

let's break it down:

- $-k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega$ is a PD action on the errors in $SO(3)$,
- $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$ to cancel the gyroscopic terms,
- $J(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d)$ is a feedforward action of the desired angular acceleration.

The closed loop dynamics achieved with this choice of $\boldsymbol{\Gamma}$ is the following:

$$J\dot{\mathbf{e}}_\Omega = -k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega \tag{2.218}$$

$$\dot{\mathbf{e}}_R = C(R_d^\top R)\mathbf{e}_\Omega \tag{2.219}$$

$$\frac{d}{dt}(R_d^\top R) = R_d^\top R S(\mathbf{e}_\Omega) \tag{2.220}$$

Observation 2.3.2

The closed loop attitude dynamics is **autonomous** in the variables $(R_d^\top R, \mathbf{e}_\Omega)$: it does not depend on the translational motion of the vehicle, nor on the thrust T. This is what allows us to analyze the complete system as a *cascade*: the attitude error subsystem is studied first, and then it is seen as an input for the position error subsystem.

Note that the hierarchical structure (outer loop for the position, inner loop for the attitude) is the same of the previous controllers, but here **no time-scale separation is assumed**: the stability will be proved for the complete closed loop.

2.3.1 Stability of the Attitude Dynamics

We study the stability of the attitude error dynamics with the following Lyapunov candidate, that combines the kinetic energy of the angular velocity error and the attitude error function:

$$V = \frac{1}{2}\mathbf{e}_\Omega^\top J \mathbf{e}_\Omega + k_R \Psi(R, R_d). \tag{2.221}$$

By using the closed loop dynamics and $\dot{\Psi} = \mathbf{e}_R^\top \mathbf{e}_\Omega$ we get:

$$\begin{aligned}
\dot{V} &= \mathbf{e}_\Omega^\top J \dot{\mathbf{e}}_\Omega + k_R \dot{\Psi} && (2.222)\\
&= \mathbf{e}_\Omega^\top (-k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega) + k_R \mathbf{e}_R^\top \mathbf{e}_\Omega && (2.223)\\
&= -k_\Omega \|\mathbf{e}_\Omega\|^2 \leq 0. && (2.224)
\end{aligned}$$

$\dot{V}$ is only negative semidefinite, but it is sufficient to prove an important property: V is non-increasing along the trajectories, hence

$$k_R \Psi(t) \leq V(t) \leq V(0). \tag{2.225}$$

If the initial condition is such that $V(0) < 2k_R$, then $\Psi(t) < 2$ for all $t \geq 0$: **the attitude error never reaches the maximum value** 2, i.e. the 180° error (the undesired equilibria, where $\mathbf{e}_R = \mathbf{0}$ but $R \neq R_d$). We can write the condition on $V(0)$ as a condition on the initial angular velocity error. Since $V(0) \leq \frac{1}{2}\lambda_{\max}(J)\|\mathbf{e}_\Omega(0)\|^2 + k_R \Psi(0)$, the condition $V(0) < 2k_R$ holds if

$$\|\mathbf{e}_\Omega(0)\|^2 < \frac{2k_R}{\lambda_{\max}(J)}\left(2 - \Psi(0)\right) \tag{2.226}$$

where $\lambda_{\max}(J)$ is the maximum eigenvalue of J (and $\lambda_{\min}(J)$ will be the minimum one).

To prove *exponential* stability we need a strict Lyapunov function, that we obtain by adding a cross term to V, with $c_2 > 0$ a constant to be chosen:

$$V_2 = \frac{1}{2}\mathbf{e}_\Omega^\top J \mathbf{e}_\Omega + k_R \Psi + c_2 \mathbf{e}_R^\top \mathbf{e}_\Omega. \tag{2.227}$$

The derivative of the cross term is

$$\begin{aligned}
\frac{d}{dt}\left(\mathbf{e}_R^\top \mathbf{e}_\Omega\right) &= \dot{\mathbf{e}}_R^\top \mathbf{e}_\Omega + \mathbf{e}_R^\top \dot{\mathbf{e}}_\Omega && (2.228)\\
&= \left(C(R_d^\top R)\mathbf{e}_\Omega\right)^\top \mathbf{e}_\Omega - \mathbf{e}_R^\top J^{-1}(k_R \mathbf{e}_R + k_\Omega \mathbf{e}_\Omega) && (2.229)\\
&\leq \|\mathbf{e}_\Omega\|^2 - \frac{k_R}{\lambda_{\max}(J)}\|\mathbf{e}_R\|^2 + \frac{k_\Omega}{\lambda_{\min}(J)}\|\mathbf{e}_R\|\|\mathbf{e}_\Omega\| && (2.230)
\end{aligned}$$

where we used $\|C\| \leq 1$. By summing it (multiplied by c_2) to $\dot{V}$ we obtain:

$$\dot{V}_2 \leq -(k_\Omega - c_2)\|\mathbf{e}_\Omega\|^2 - \frac{c_2 k_R}{\lambda_{\max}(J)}\|\mathbf{e}_R\|^2 + \frac{c_2 k_\Omega}{\lambda_{\min}(J)}\|\mathbf{e}_R\|\|\mathbf{e}_\Omega\| = -\mathbf{z}_2^\top W_2 \mathbf{z}_2 \tag{2.231}$$

with

$$\mathbf{z}_2 = \begin{pmatrix} \|\mathbf{e}_R\| \\ \|\mathbf{e}_\Omega\| \end{pmatrix}, \quad W_2 = \begin{pmatrix} \frac{c_2 k_R}{\lambda_{\max}(J)} & -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} \\ -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} & k_\Omega - c_2 \end{pmatrix}. \tag{2.232}$$

W_2 is positive definite if c_2 is small enough. Furthermore, on the sublevel set $\{\Psi \leq \psi_2 < 2\}$ the bounds in Equation (2.201) hold, hence V_2 is bounded between two quadratic forms of $\mathbf{z}_2$ (if c_2 is small enough), and $\dot{V}_2 \leq -\lambda_{\min}(W_2)\|\mathbf{z}_2\|^2$. This proves the following result.

Proposition 2.3.2 : Exponential Stability of the Attitude Dynamics

Consider the attitude tracking controller $\boldsymbol{\Gamma}$ defined above. If the initial errors satisfy

$$\Psi(R(0), R_d(0)) < 2 \tag{2.233}$$

$$\|\mathbf{e}_\Omega(0)\|^2 < \frac{2k_R}{\lambda_{\max}(J)}\left(2 - \Psi(R(0), R_d(0))\right) \tag{2.234}$$

then the zero equilibrium of the attitude tracking errors $(\mathbf{e}_R, \mathbf{e}_\Omega)$ is **exponentially stable**: $\mathbf{e}_R, \mathbf{e}_\Omega \to \mathbf{0}$ exponentially.

In words: the attitude error converges exponentially to zero for any initial attitude error smaller than 180° ($\Psi < 2$), provided that the initial angular velocity error is not too large.

2.3.2 Position Error Dynamics

We now study the translational dynamics. Recall that $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d$ and $\mathbf{e}_v = \dot{\mathbf{p}} - \dot{\mathbf{p}}_d$, hence

$$m\dot{\mathbf{e}}_v = mg\mathbf{e}_3 - TR\mathbf{e}_3 - m\ddot{\mathbf{p}}_d. \tag{2.235}$$

If the thrust force $-TR\mathbf{e}_3$ were exactly equal to the positional control action $\mathbf{a}$, we would get $m\dot{\mathbf{e}}_v = -k_p\mathbf{e}_p - k_d\mathbf{e}_v$, i.e. a stable linear error dynamics. Since this is not the case when $\mathbf{b}_3 \neq \mathbf{b}_{3d}$, we add and subtract the force along the desired direction $\mathbf{b}_{3d}$:

$$-TR\mathbf{e}_3 = -\frac{T}{\mathbf{b}_{3d}^\top\mathbf{b}_3}\mathbf{b}_{3d} - \mathbf{X}, \qquad \mathbf{X} := \frac{T}{\mathbf{b}_{3d}^\top\mathbf{b}_3}\left((\mathbf{b}_{3d}^\top\mathbf{b}_3)\mathbf{b}_3 - \mathbf{b}_{3d}\right). \tag{2.236}$$

Since $T = \|\mathbf{a}\|(\mathbf{b}_{3d}^\top\mathbf{b}_3)$ and $-\|\mathbf{a}\|\mathbf{b}_{3d} = \mathbf{a}$, the first term is exactly $\mathbf{a}$, and the position error dynamics becomes:

$$m\dot{\mathbf{e}}_v = -k_p\mathbf{e}_p - k_d\mathbf{e}_v - \mathbf{X}. \tag{2.237}$$

The vector $\mathbf{X}$ is the effect of the attitude error on the translational dynamics. Some properties:

- $\mathbf{X} = \mathbf{0}$ when $\mathbf{b}_3 = \mathbf{b}_{3d}$: **if there is no attitude error, the position error converges** (linear dynamics in Equation (2.237)).
- Let α be the angle between $\mathbf{b}_3$ and $\mathbf{b}_{3d}$. Then $\|(\mathbf{b}_{3d}^\top\mathbf{b}_3)\mathbf{b}_3 - \mathbf{b}_{3d}\| = \sin\alpha$ and, since the thrust axes are the third columns of R and R_d, $\alpha \leq \theta$ where θ is the angle of the relative rotation $R_d^\top R$.
- For $\Psi < 1$ ($\theta < \frac{\pi}{2}$) it holds $\sin\alpha \leq \sin\theta = \|\mathbf{e}_R\|$, and $\|\mathbf{a}\| \leq k_p\|\mathbf{e}_p\| + k_d\|\mathbf{e}_v\| + \mathcal{B}$, hence

  $$\|\mathbf{X}\| = \|\mathbf{a}\|\sin\alpha \leq \left(k_p\|\mathbf{e}_p\| + k_d\|\mathbf{e}_v\| + \mathcal{B}\right)\|\mathbf{e}_R\|. \tag{2.238}$$

  where $\mathcal{B}$ is the bound on the reference, $\|-mg\mathbf{e}_3 + m\ddot{\mathbf{p}}_d\| \leq \mathcal{B}$ (we use $\mathcal{B}$ in order to avoid confusion with the matrix B of the DFL). Moreover $\|\mathbf{e}_R\| \leq \sqrt{\psi_1(2-\psi_1)} < 1$ on the set $\{\Psi \leq \psi_1 < 1\}$.
- For $\Psi < 1$ we have also $\alpha < \frac{\pi}{2}$, hence $T = \|\mathbf{a}\|\cos\alpha > 0$: the dot product in the definition of T **reduces the thrust when the two axes separate**, and keeps it positive in the region where the analysis of the complete system holds.

The assumptions are $\|\mathbf{a}\| \neq 0$ (no free fall, the analogue of $T \neq 0$ in the DFL) and a bounded reference $\|-mg\mathbf{e}_3 + m\ddot{\mathbf{p}}_d\| \leq \mathcal{B}$.

2.3.3 Stability of the Complete System

The structure of the complete error dynamics is a cascade: the attitude error subsystem $(\mathbf{e}_R, \mathbf{e}_\Omega)$ is autonomous, and it drives the position error subsystem $(\mathbf{e}_p, \mathbf{e}_v)$ through the term $\mathbf{X}$. For the position part we choose the following Lyapunov function, with $c_1 > 0$ a constant to be chosen:

$$V_1 = \frac{1}{2}k_p\|\mathbf{e}_p\|^2 + \frac{1}{2}m\|\mathbf{e}_v\|^2 + c_1\mathbf{e}_p^\top\mathbf{e}_v \tag{2.239}$$

and for the complete system $V = V_1 + V_2$. By deriving V_1 along Equation (2.237) we get:

$$\dot{V}_1 \leq -\frac{c_1k_p}{m}\|\mathbf{e}_p\|^2 - (k_d - c_1)\|\mathbf{e}_v\|^2 + \frac{c_1k_d}{m}\|\mathbf{e}_p\|\|\mathbf{e}_v\| + \|\mathbf{X}\|\left(\|\mathbf{e}_v\| + \frac{c_1}{m}\|\mathbf{e}_p\|\right). \tag{2.240}$$

The first three terms are the ones of the nominal system (without attitude error). The last term is bounded with Equation (2.238), and it is a term of the form $\|\mathbf{e}_R\| \cdot (\dots)$. On the domain $\|\mathbf{e}_p\| < e_{p,\max}$ the bound is a quadratic form, and we obtain:

$$\dot{V} \leq -\mathbf{z}_1^\top W_1\mathbf{z}_1 + \mathbf{z}_1^\top W_{12}\mathbf{z}_2 - \mathbf{z}_2^\top W_2\mathbf{z}_2, \qquad \mathbf{z}_1 = \begin{pmatrix}\|\mathbf{e}_p\| \\ \|\mathbf{e}_v\|\end{pmatrix} \tag{2.241}$$

with

$$W_1 = \begin{pmatrix}\frac{c_1k_p}{m} & -\frac{c_1k_d}{2m} \\ -\frac{c_1k_d}{2m} & k_d - c_1\end{pmatrix} \tag{2.242}$$

and W_{12} collects the bound on $\mathbf{X}$ (it depends on $\mathcal{B}$, $e_{p,\max}$, k_p, k_d and c_1). We have

$$\dot{V} \leq -\lambda_{\min}(W_1)\|\mathbf{z}_1\|^2 + \|W_{12}\|\|\mathbf{z}_1\|\|\mathbf{z}_2\| - \lambda_{\min}(W_2)\|\mathbf{z}_2\|^2 \tag{2.243}$$

that is negative definite if and only if (discriminant of the quadratic form):

$$4\lambda_{\min}(W_1)\lambda_{\min}(W_2) > \|W_{12}\|^2. \tag{2.244}$$

This is a condition on the attitude gains k_R, k_Ω (that appear in W_2) with respect to the position gains: **the attitude gains must be large enough with respect to the position loop**. It is a provable version of the rule "the inner loop must be faster than the outer loop" that we mentioned for the hierarchical control scheme.

Proposition 2.3.3 : Exponential Stability of the Complete System

For any positive k_p, k_d there exist c_1, c_2, k_R, k_Ω such that, if

$$\Psi(R(0), R_d(0)) < \psi_1 < 1 \tag{2.245}$$
$$\|\mathbf{e}_p(0)\| < e_{p,\max} \tag{2.246}$$
$$\|\mathbf{e}_\Omega(0)\| \text{ is small enough} \tag{2.247}$$

the zero equilibrium of the tracking errors $(\mathbf{e}_p, \mathbf{e}_v, \mathbf{e}_R, \mathbf{e}_\Omega)$ of the complete system is **exponentially stable**. This requires an initial attitude error smaller than 90°.

The region of Proposition 2.3.3 is smaller than the one of Proposition 2.3.2. Let's see how we can extend the result to initial attitude errors up to 180°.

Proposition 2.3.4 : Almost Global Exponential Attractiveness

If $\Psi(R(0), R_d(0)) < 2$ and

$$\|\mathbf{e}_\Omega(0)\|^2 < \frac{2k_R}{\lambda_{\max}(J)}\left(2 - \Psi(R(0), R_d(0))\right) \tag{2.248}$$

then the zero equilibrium of the tracking errors of the complete system is **exponentially attractive**.

The idea of the proof is the following:

1. Since the attitude subsystem is autonomous, from any $\Psi(0) < 2$ (with the bound on $\mathbf{e}_\Omega(0)$) the attitude error decreases exponentially and enters the set $\{\Psi \leq \psi_1\}$ at a finite time t^*.
2. On the interval $[0, t^*]$ the position errors remain bounded: $\|\mathbf{X}\|$ grows at most linearly with $\|\mathbf{e}_p\|, \|\mathbf{e}_v\|$ (by Equation (2.238) with $\sin\alpha \leq 1$), hence the system of Equation (2.237) is a linear system with a bounded-gain input, and there is **no finite escape time**.
3. After t^*, Proposition 2.3.3 applies.

L_2 almost covers $SO(3)$, hence this is an **almost global exponential attractiveness**, that is the best result achievable with a continuous feedback law, because of the topological obstruction. The following table summarizes the stability properties of the controller:

initial attitude error	subsystem	property
$\Psi < 2$ ($\theta < 180°$)	attitude	exponential stability
$\Psi \leq \psi_1 < 1$ ($\theta < 90°$)	complete	exponential stability
$\Psi < 2$ ($\theta < 180°$)	complete	exponential attractiveness

Observation 2.3.3

Almost global convergence is a statement about the *basin of attraction*, not about the *performance*. Near the 180° error we have $\|\mathbf{e}_R\| = \sin\theta \simeq 0$, hence the torque $-k_R\mathbf{e}_R$ is very small and the error leaves this region very slowly.

2.3.4 Computation of the Desired Angular Velocity and Acceleration

The attitude controller requires $\boldsymbol{\Omega}_d$ and $\dot{\boldsymbol{\Omega}}_d$, that are defined through the desired attitude $R_d(t)$:

$$S(\boldsymbol{\Omega}_d) = R_d^\top \dot{R}_d \tag{2.249}$$

$$S(\dot{\boldsymbol{\Omega}}_d) = R_d^\top \ddot{R}_d - S(\boldsymbol{\Omega}_d)^2. \tag{2.250}$$

The second equation follows from the derivative of the first one: $\frac{d}{dt}(R_d^\top \dot{R}_d) = \dot{R}_d^\top \dot{R}_d + R_d^\top \ddot{R}_d$, and $\dot{R}_d^\top \dot{R}_d = S(\boldsymbol{\Omega}_d)^\top R_d^\top R_d S(\boldsymbol{\Omega}_d) = -S(\boldsymbol{\Omega}_d)^2$.

R_d depends on $\mathbf{b}_{1d}(t)$ and on $\mathbf{a}(\mathbf{e}_p, \mathbf{e}_v, \ddot{\mathbf{p}}_d)$, hence we have to compute the derivatives of $\mathbf{a}$. We want to avoid the numerical differentiation of the measurements, so we use the model:

$$\dot{\mathbf{e}}_v = g\mathbf{e}_3 - \frac{T}{m}R\mathbf{e}_3 - \ddot{\mathbf{p}}_d \tag{2.251}$$

$$\dot{\mathbf{a}} = -k_p\mathbf{e}_v - k_d\dot{\mathbf{e}}_v + m\mathbf{p}_d^{(3)} \tag{2.252}$$

$$\dot{T} = -\dot{\mathbf{a}}^\top R\mathbf{e}_3 - \mathbf{a}^\top RS(\boldsymbol{\Omega})\mathbf{e}_3 \tag{2.253}$$

$$\ddot{\mathbf{e}}_v = -\frac{\dot{T}}{m}R\mathbf{e}_3 - \frac{T}{m}RS(\boldsymbol{\Omega})\mathbf{e}_3 - \mathbf{p}_d^{(3)} \tag{2.254}$$

$$\ddot{\mathbf{a}} = -k_p\dot{\mathbf{e}}_v - k_d\ddot{\mathbf{e}}_v + m\mathbf{p}_d^{(4)}. \tag{2.255}$$

where $\dot{T}$ comes from the derivative of $T = -\mathbf{a}^\top R\mathbf{e}_3$. Now we need the derivatives of $\mathbf{b}_{3d}$, $\mathbf{b}_{2d}$ and then of R_d. Both are obtained by normalizing a vector. For a generic vector $\mathbf{h}$ and the unit vector $\mathbf{s} = \frac{\mathbf{h}}{\|\mathbf{h}\|}$:

$$\dot{\mathbf{s}} = \frac{\dot{\mathbf{h}} - \mathbf{s}(\mathbf{s}^\top\dot{\mathbf{h}})}{\|\mathbf{h}\|} \tag{2.256}$$

$$\ddot{\mathbf{s}} = \frac{\ddot{\mathbf{h}} - 2\dot{\mathbf{s}}(\mathbf{s}^\top\dot{\mathbf{h}}) - \mathbf{s}(\dot{\mathbf{s}}^\top\dot{\mathbf{h}} + \mathbf{s}^\top\ddot{\mathbf{h}})}{\|\mathbf{h}\|}. \tag{2.257}$$

We apply these formulas first to $\mathbf{b}_{3d}$ (with $\mathbf{h} = -\mathbf{a}$), then to $\mathbf{b}_{2d}$ (with $\mathbf{h} = \mathbf{b}_{3d} \times \mathbf{b}_{1d}$), and the product rule on $\mathbf{b}_{2d} \times \mathbf{b}_{3d}$, which gives $\dot{R}_d$ and $\ddot{R}_d$.

- This requires $\mathbf{p}_d \in C^4$ (up to the snap) and $\mathbf{b}_{1d} \in C^2$: the same smoothness required for the flat outputs in the DFL.
- $\ddot{\mathbf{a}}$ depends on the state (through $\boldsymbol{\Omega}$) but not on the control torque $\boldsymbol{\Gamma}$: there is no algebraic loop.
- In many implementations $\dot{\boldsymbol{\Omega}}_d$ is neglected or obtained through a filtered differentiation: in this case the exact cancellation of the feedforward term, and hence the proof, hold only approximately.

Let's discuss about some *implementation remarks*:

- **Sign of the thrust**: $T = \|\mathbf{a}\| \cos\alpha < 0$ when the thrust axis is more than $90°$ away from $\mathbf{b}_{3d}$, hence the control law assumes a *reversible* thrust. With real propellers T is saturated in $[0, T_{\max}]$; in the region $\{\Psi < 1\}$ we have $T > 0$ anyway. Since the attitude error dynamics do not depend on T, the attitude still converges, and the position recovers afterwards.
- **Singular situations** appear only in the reference generation:
  - $\|\mathbf{a}\| = 0$: commanded free fall, $\mathbf{b}_{3d}$ is undefined (the analogue of $T = 0$ in the DFL);
  - $\mathbf{b}_{3d} \parallel \mathbf{b}_{1d}$: the desired heading is undefined, it can be avoided by changing $\mathbf{b}_{1d}$.
- The controller is a **static** feedback: there is no compensator state to initialize (in the DFL we have $\zeta = (T, \dot{T})$).
- Numerical example: $m = 4.34\,\text{kg}$, $J = \text{diag}(0.082, 0.0845, 0.1377)\,\text{kg m}^2$, $k_p = 16m$, $k_d = 5.6m$, $k_R = 8.81$, $k_\Omega = 2.54$.

2.3.5 Comparison Between DFL and Geometric Control

We compare the complete DFL controller with the geometric controller on $SE(3)$ in simulation, by using the same trajectories and initial conditions.

Simulation 1: tracking of an 8-shaped trajectory while spinning, i.e. far from the singularities:

$$\mathbf{p}_d = \begin{pmatrix} 2\sin\frac{2\pi}{T}t & \sin 2\frac{2\pi}{T}t & 0.6\sin\frac{2\pi}{T}t + \sin 2\frac{2\pi}{T}t \end{pmatrix}^\top, \qquad \psi_d = \pi \sin\frac{2\pi}{T}t \tag{2.258}$$

(here T is the period of the trajectory, not the thrust). The attitude error function of the $SE(3)$ controller is $\Psi(R, R_d) = \frac{1}{2}\text{tr}\left[I - R_d^\top R\right]$, and it converges to zero in about one second.

- The two controllers show different transients in position tracking, then both converge to the reference.
- The DFL has a faster convergence for a comparable control effort.
- The $SE(3)$ controller tracks only the *projection* of $\mathbf{b}_{1d}$ on the plane orthogonal to $\mathbf{b}_{3d}$: its RPY yaw angle differs from ψ_d when the vehicle is tilted. The DFL tracks exactly the yaw angle ψ_d.

Simulation 2: tracking of a helix with a constant yaw angle:

$$\mathbf{p}_d = \begin{pmatrix} 0.2t & \sin\frac{2\pi}{T}t & \cos\frac{2\pi}{T}t \end{pmatrix}^\top, \qquad \psi_d = 0. \tag{2.259}$$

- The complete DFL **runs into the singularity** $\vartheta = -\frac{\pi}{2}$ at $t \simeq 0.72$ s.
- Here $\det M(\boldsymbol{\Theta}) = \cos\vartheta \to 0$: this is the gimbal lock of the RPY angles. The yaw row of the matrix D contains $\frac{1}{c_\vartheta}$, hence the control law is not defined and the angular rates diverge.
- With the same trajectory and the same initial conditions, the $SE(3)$ controller converges: $\Psi \to 0$ in about 1 s, and the position error converges after the transient.

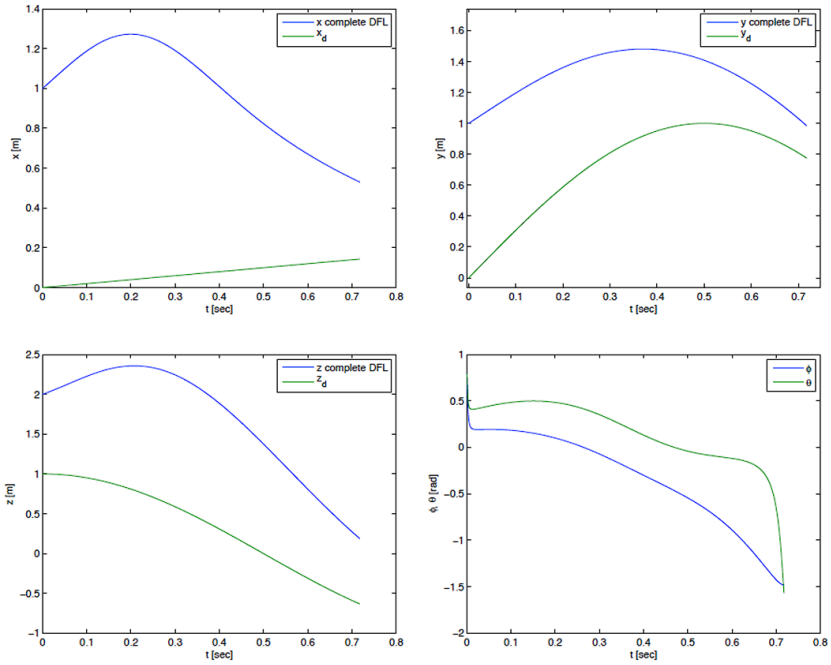


Figure 2.5: Simulation 2: complete DFL (singularity at $t \simeq 0.72$ s) and $SE(3)$ controller.

The following table compares the two controllers:

	complete DFL	**geometric control on** $SE(3)$
attitude	RPY angles (local)	rotation matrix (global)
feedback	dynamic, $\zeta = (T, \dot{T})$	static
outputs	(x, y, z, ψ)	$\mathbf{p}$ and heading $\mathbf{b}_{1d}$ (projection)
closed loop	linear, decoupled	nonlinear, Lyapunov-based
convergence	exponential, away from the singularities	exponential for $\Psi < 1$, almost global attractiveness
singularities	$T = 0$, $\varphi = \pm\frac{\pi}{2}$, $\vartheta = \pm\frac{\pi}{2}$	$\|\mathbf{a}\| = 0$, $\mathbf{b}_{3d} \parallel \mathbf{b}_{1d}$ (reference only)
reference	C^4 position, C^2 yaw	C^4 position, C^2 heading

To summarize:

- Due to the occurrence of singularities in the DFL, the $SE(3)$ controller is more **reliable**, in particular for large attitude excursions.
- When no singularity occurs, no significant difference can be appreciated between the two controllers.
- The control effort of the DFL becomes huge when the determinant of the decoupling matrix D approaches 0 (or diverges, at $\vartheta = \pm\frac{\pi}{2}$).
- The complete DFL is a linear controller for a fully linearized system: far from the singularities the convergence is exponential. The convergence of the complete dynamics of the $SE(3)$ controller becomes exponential only when the attitude error is below 90°.
- Both controllers need the reference up to the snap and an accurate model of the vehicle.

CHAPTER

3

STATE ESTIMATION

3.1 Sensors

The controllers that we have seen in Chapter 2 use the state $\mathbf{p}$, $\dot{\mathbf{p}}$, R, $\boldsymbol{\Omega}$ to compute the control inputs, none of them is measured directly and exactly by the onboard sensors. Sensors measures other quantities (subject to bias and noise), that are used to reconstruct the state. Some of the most known sensors are the following:

- **Barometer**: measures the height and its variation from the air pressure.
- **Three-axis magnetic compass**: measures the Earth magnetic field and provides the orientation with respect to the (magnetic) north.
- **Inertial Measurement Unit (IMU)**: an electronic device that measures and transmits linear acceleration and orientation (roll, pitch, yaw).
- **Sonar, Laser, Infrared**: to measure the distance from obstacles, including the ground.
- **Monocular camera**: for vision-based navigation.
- **GPS**: absolute position outdoors.

sensor	measures	frame	typical rate	main issues
rate gyro	angular velocity Ω	SR_B	0.1–1 kHz	bias, drift if integrated
accelerometer	specific force	SR_B	0.1–1 kHz	bias, vibrations
magnetometer	magnetic field	SR_B	10–100 Hz	motors, metal nearby
barometer	pressure $\rightarrow$ height	SR_I	10–100 Hz	drift, propeller wash
GPS	position (velocity)	SR_I	1–10 Hz	meters, outdoors only
camera	features, rel. pose	SR_B	20–60 Hz	computation, scale
sonar / laser	distance	SR_B	10–50 Hz	limited range
motion capture	pose	SR_I	0.1–0.5 kHz	indoor lab only

We recall that with SR_I we denote the inertial frame,with its z-axis pointing downward (also denoted $\mathbf{e}_3$), and SR_B the body frame, specified by the rotation matrix R. In this chapter, in the translational

dynamics we will include the drag force:

$$m\ddot{\mathbf{p}} = mg\mathbf{e}_3 - TR\mathbf{e}_3 - TRDR^\top \dot{\mathbf{p}} \tag{3.1}$$

with D being a diagonal matrix:

$$D = \begin{pmatrix} d_x & 0 & 0 \\ 0 & d_y & 0 \\ 0 & 0 & 0 \end{pmatrix} \tag{3.2}$$

the drag is proportional to the thrust and the velocity of the body. It is small, often neglected in control design, but it is what makes the accelerometer informative.

The **barometer** measures the air pressure, it provides a measurement of the height and its variation. We model the quantity provided by the barometer as follows:

$$\tilde{h} = h + d + \eta \tag{3.3}$$

h is the real height of the body, expressed in SR_I, d is a disturbance, that is usually slow varying, and η is the measurements noise.

The **magnetometer** provides a measure of the Earth magnetic field:

$$\mathbf{m}_{IMU} = R^\top {}^I\mathbf{m} + \mathbf{B}_m + \boldsymbol{\eta}_m \tag{3.4}$$

where

- ${}^I\mathbf{m}$ is the real Earth magnetic field in the inertial frame (the provided quantity is in the body frame).
- $\mathbf{B}_m$ is the local magnetic disturbance due to ferromagnetic components and motor currents.
- $\boldsymbol{\eta}_m$ is the measurements noise.

The **rate gyro** measures the angular velocity of the body with respect to the inertial frame, expressed in the body frame:

$$\boldsymbol{\Omega}_{IMU} = \boldsymbol{\Omega} + \mathbf{b}_\Omega + \boldsymbol{\eta} \tag{3.5}$$

where

- $\mathbf{b}_\Omega$ is a constant or slowly time-varying bias.
- $\boldsymbol{\eta}$ is the measurements noise.

The **accelerometer** measures the instantaneous linear acceleration of the body frame due to exogenous forces, excluding gravity:

$$\mathbf{a}_{IMU} = R^\top(\ddot{\mathbf{p}} - g\mathbf{e}_3) + \mathbf{b}_a + \boldsymbol{\eta}_a \tag{3.6}$$

where $\mathbf{b}_a$ is a constant or slowly time-varying bias and $\boldsymbol{\eta}_a$ the measurements noise. The measured acceleration can be approximated as

$$\mathbf{a}_{IMU} = -\frac{T}{m}\mathbf{e}_3 - \frac{T}{m}DR^\top \dot{\mathbf{p}} \tag{3.7}$$

where $-\frac{T}{m}DR^\top \dot{\mathbf{p}}$ is the acceleration due to the blade flapping and induced drag.

3.2 Attitude Estimation

We describe the process of estimating the attitude for one axis, consider the pitch ϑ (the same reasoning applies for the roll and yaw too), let ω_g be the angular velocity measured by the gyroscope:

$$\omega_g = \dot{\vartheta} + b + \eta \tag{3.8}$$

with b being the bias and η the measurement noise. Let a_y, a_z being the components of the *measured acceleration* expressed in the body frame, we denote ϑ_a the angle estimated directly from the accelerometer

$$\vartheta_a = \text{atan2}(-a_y, -a_z) \tag{3.9}$$

. It is based on the hypothesis that at rest the accelerometer only measures the reaction to gravity. We denote the **estimation of the pitch** $\hat{\vartheta}$ and we define its dynamic as follows:

$$\dot{\hat{\vartheta}} = \omega_g - \hat{b} + k_P(\vartheta_a - \hat{\vartheta}). \tag{3.10}$$

The derivative of the estimated angle is obtained by integrating the bias-corrected gyroscope reading $(\omega_g - \hat{b})$, to which is added a correction term proportional to the error between the accelerometer measurement and the current estimate $(\vartheta - \hat{\vartheta})$. $\hat{b}$ is the estimation of the bias, with the following dynamics:

$$\dot{\hat{b}} = -k_I(\vartheta_a - \hat{\vartheta}) \tag{3.11}$$

if a constant error persists between the accelerometer and the estimate, this is attributed to the gyroscope bias and integrated over time to zero the steady-state error. k_P, k_I are positive gains.

Bringing the equations into the Laplace domain and eliminating the intermediate variable $\hat{b}(s)$, we obtain:

$$\hat{\vartheta}(s) = \underbrace{\frac{k_P s + k_I}{s^2 + k_P s + k_I}}_{G_{\text{low}}(s)} \vartheta(s) + \underbrace{\frac{s^2}{s^2 + k_P s + k_I}}_{G_{\text{high}}(s)} \frac{\omega_g(s)}{s} \tag{3.12}$$

Noting that integrating the angular velocity corresponds to $\frac{\omega_g(s)}{s}$:

- Low-pass filter (G_{low}): acts on the accelerometer measurement $\theta_a(s)$, cutting out vibrations and high-frequency dynamic accelerations.
- High-pass filter (G_{high}): acts on the angle integrated by the gyroscope $\frac{\omega_g(s)}{s}$, blocking the zero/very low frequency component responsible for the drift.

The two filters are complementary because their algebraic sum is exactly 1:

$$G_{\text{low}}(s) + G_{\text{high}}(s) = \frac{k_P s + k_I + s^2}{s^2 + k_P s + k_I} = 1. \tag{3.13}$$

About the choice of the gains:

- Crossover frequency: determined by the values of k_p and $\sqrt{k_I}$. It establishes the border point: below this frequency we rely on the accelerometer, above it we rely on the gyroscope.
- If $k_I = 0$ (without bias estimate), a constant gyroscope bias b generates a steady state error equal to $\frac{b}{k_P}$.
- With $k_I > 0$, the integral action fully compensates for the steady-state bias, bringing the static error to zero.

Now let's see how to define an observer that tries to estimate the full attitude matrix $R \in SO(3)$, we denote $\hat{R}$ the estimated rotation matrix. To estimate the full 3D orientation (including yaw), the accelerometer alone is no longer enough (which only identifies the axis of gravity, leaving an undetermined degree of freedom around the vertical). We need at least two known non-parallel directions in inertial space:

- The gravity vector $\mathbf{e}_3$.
- The Earth magnetic field ${}^I\mathbf{m}$, known and fixed in the inertial frame, we use the normalized vector $\bar{\mathbf{m}} = \frac{{}^I\mathbf{m}}{\|{}^I\mathbf{m}\|}$.

For each of the two vectors we compare:

- The direction measured in the body:
  - $\mathbf{v}_a = -\frac{\mathbf{a}_{IMU}}{g} \simeq R^\top \mathbf{e}_3$ (measured by normalized accelerometer, at rest).

  - $\mathbf{v}_m = {}^{\mathbf{m}_{IMU}}/_{\|{}^I\mathbf{m}\|} \simeq R^\top \bar{\mathbf{m}}$ (measured by normalized magnetometer).

- The direction predicted by the current attitude estimate $\hat{R}$:
  - $\hat{\mathbf{v}}_a = \hat{R}^\top \mathbf{e}_3$ (where we should see the gravity according to the current estimate).
  - $\hat{\mathbf{v}}_m = \hat{R}^\top \bar{\mathbf{m}}$ (where we should see magnetic north according to the current estimate).

We define the orientation reconstruction error $\boldsymbol{\alpha}$ as follows:

$$\boldsymbol{\alpha} = k_a(\hat{\mathbf{v}}_a \times \mathbf{v}_a) + k_m(\hat{\mathbf{v}}_m \times \mathbf{v}_m). \tag{3.14}$$

k_a and k_m are the gains that weight the reliability of the accelerometer and magnetometer. If desired, further known directions provided by other sensors (e.g. cameras, star sensors) can be added. The dynamics of the observer is defined as follows:

$$\dot{\hat{R}} = \hat{R}\, S(\boldsymbol{\Omega}_{IMU} - \hat{\mathbf{b}} - \boldsymbol{\alpha}) \tag{3.15}$$

we are using the skew-symmetric matrix, and $\hat{\mathbf{b}}$ is the estimation on the bias with the following dynamics:

$$\dot{\hat{\mathbf{b}}} = k_b \boldsymbol{\alpha} \tag{3.16}$$

acting as an integral action. The logic exactly reflects the scalar filter (one axis attitude), but extended geometrically to $SO(3)$.

3.3 Velocity Estimation

Normally to calculate the velocity one would have to integrate the acceleration over time (accumulating drift), but in some cases the velocity is obtained algebraically instant by instant. We work in the *hypothesis* of *horizontal flight*: $T \simeq mg$ (the total thrust balances gravity). The measurement of the accelerometer considering the drag of the rotors is:

$$\mathbf{a}_{IMU} = -\frac{T}{m}\mathbf{e}_3 - \frac{T}{m} D R^\top \dot{\mathbf{p}} = -\frac{T}{m} \begin{pmatrix} d_x \dot{p}_{B,x} \\ d_y \dot{p}_{B,y} \\ 1 \end{pmatrix} \tag{3.17}$$

where $R^\top \dot{\mathbf{p}} = \begin{pmatrix} \dot{p}_{B,x} & \dot{p}_{B,y} & \dot{p}_{B,z} \end{pmatrix}^\top$ is the linear velocity expressed in the drone's coordinate system (body frame). Bringing the accelerometer measurement into the inertial frame via the estimated rotation matrix $\hat{R} \simeq R$:

$${}^I\mathbf{a}_{IMU} = R\mathbf{a}_{IMU} \simeq \hat{R} a_{IMU} \simeq -g\hat{R}\mathbf{e}_3 - g\hat{R}D\hat{R}^\top \dot{\mathbf{p}} \tag{3.18}$$

given the assumption, $\frac{T}{m}$ was replaced with g. Going back to working directly in body frame:

$$\mathbf{a}_{\text{IMU}} \simeq -g\mathbf{e}_3 - gDR^\top \dot{\mathbf{p}} \implies \mathbf{a}_{\text{IMU}} + g\mathbf{e}_3 \simeq -gDR^\top \dot{\mathbf{p}}. \tag{3.19}$$

Expliciting the three rows of this vector:

- First row (x): $a_x = -g\, d_x \dot{p}_{B,x}$
- Second row (y): $a_y = -g\, d_y \dot{p}_{B,y}$
- Third row (z): $a_z + g \simeq 0 \implies a_z \simeq -g$ (confirms that along z the accelerometer only measures the reaction to the hover weight).

By isolating $\dot{p}_{B,x}$ and $\dot{p}_{B,y}$ from the first two components gives the following:

$$\dot{p}_{B,x} \simeq -\frac{a_x}{g\, d_x}, \qquad \dot{p}_{B,y} \simeq -\frac{a_y}{g\, d_y}. \tag{3.20}$$

When a quadrotor moves forward or sideways, the rotating rotors generate an aerodynamic drag force in the xy plane of the body proportional to the translational velocity ($\dot{\mathbf{p}}_B$). The accelerometer senses this

specific force on the a_x and a_y components. Knowing the aerodynamic parameters d_x, d_y, the readings a_x and a_y instantly provide the speed of the drone.

If the drone is not hovering and accelerating vertically ($T \neq mg$), the approximation $a_z \simeq -g$ no longer holds. However, from the third component of the general model we have:

$$a_z = -\frac{T}{m}.$$

By directly substituting this relation, the thrust hypothesis is completely eliminated without even having to know the orientation/attitude:

$$\dot{p}_{B,x} = \frac{a_x}{d_x a_z}, \qquad \dot{p}_{B,y} = \frac{a_y}{d_y a_z}.$$

The linear velocity observer estimates the inertial velocity $\dot{\mathbf{p}}$ by combining the multirotor dynamic model prediction with a correction term based on the rotor-plane velocity measurement:

$$\ddot{\hat{\mathbf{p}}} = g\mathbf{e}_3 - g\left(\hat{R}\mathbf{e}_3 + \hat{R}D\hat{R}^\top\dot{\hat{\mathbf{p}}}\right) - k_w\hat{R}P\left(\hat{R}^\top\dot{\hat{\mathbf{p}}} - \mathbf{v}_{B,m}\right) \tag{3.21}$$

where $g\mathbf{e}_3$ denotes the gravitational acceleration, $-g\hat{R}\mathbf{e}_3$ represents the thrust vector under the near-hover horizontal flight assumption ($T \simeq mg$), and $-g\hat{R}D\hat{R}^\top\dot{\hat{\mathbf{p}}}$ accounts for the rotor drag deceleration. The corrective term acts on the estimation error relative to $\mathbf{v}_{B,m}$, which is the velocity directly inferred from the accelerometer measurements in the rotor plane. The projection matrix $P = \mathrm{diag}(1, 1, 0)$ confines the correction exclusively to the measured body x and y axes. This formulation requires an identified drag matrix D as well as an accurate attitude estimate $\hat{R}$, mirroring the predictor-corrector structure commonly employed in the prediction stage of an Extended Kalman Filter (EKF).

3.4 Height Estimation

The vertical position (height) observer fuses barometric pressure altitude with inertial vertical acceleration via a complementary filter:

$$\dot{\hat{h}} = \hat{v}_h + k_1(\tilde{h} - \hat{h}), \tag{3.22}$$

$$\dot{\hat{v}}_h = a_h + k_2(\tilde{h} - \hat{h}) \tag{3.23}$$

where $\tilde{h} = h + d + \eta$ is the barometric measurement (corrupted by a slowly varying disturbance d and noise η), and $a_h = -\left(\mathbf{e}_3^\top\hat{R}\mathbf{a}_{\mathrm{IMU}} + g\right)$ denotes the vertical acceleration obtained by projecting the accelerometer reading $\mathbf{a}_{\mathrm{IMU}}$ via the estimated attitude $\hat{R}$ and subtracting gravity. In the Laplace domain, the estimated height combines a low-pass filtering of the barometer reading with a high-pass filtering of the doubly integrated vertical acceleration:

$$\hat{h}(s) = \frac{k_1 s + k_2}{s^2 + k_1 s + k_2}\tilde{h}(s) + \frac{s^2}{s^2 + k_1 s + k_2}\frac{a_h(s)}{s^2} \tag{3.24}$$

Because the slow disturbance d passes through the low-pass filter, barometric estimation remains relative, while an additional state $\hat{b}$ can be included to estimate accelerometer bias.

3.5 Sensor Fusion: Extended Kalman Filter

The Extended Kalman Filter (EKF) is a fundamental algorithm for sensor fusion, allowing for the handling of non-linear system dynamics and measurement models. The system is described by the following equations:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{w} \sim \mathcal{N}(0, Q) \tag{3.25}$$

$$\mathbf{y}_k = \mathbf{h}(\mathbf{x}_k) + \boldsymbol{\nu}_k, \quad \boldsymbol{\nu} \sim \mathcal{N}(0, S) \tag{3.26}$$

The filter operates iteratively through two distinct phases: prediction and update.

3.5.1 Prediction Phase

This phase runs at a high frequency, typically synchronous with the IMU sampling rate (IMU rate). It propagates the state estimate $\hat{\mathbf{x}}^-$ and the error covariance matrix P^- forward in time using the non-linear model $\mathbf{f}(\hat{\mathbf{x}}, \mathbf{u})$ and its Jacobian $F = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}$. In this architecture, the IMU measurements act as the control input $\mathbf{u}$ of the prediction model rather than as standard measurements:

$$\hat{\mathbf{x}}^- = \mathbf{f}(\hat{\mathbf{x}}, \mathbf{u}) \tag{3.27}$$

$$P^- = FPF^\top + Q \tag{3.28}$$

3.5.2 Update Phase

When external measurements $\mathbf{y}$ arrive at a lower frequency (such as data from GPS or vision systems), the filter performs the correction. The Kalman gain K is computed using the Jacobian of the measurement model $H = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}$ and the measurement noise covariance matrix S. This gain optimally weights the measurement residual to correct the state estimate and update the covariance P:

$$K = P^- H^\top \left(HP^- H^\top + S \right)^{-1} \tag{3.29}$$

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K \left(\mathbf{y} - \mathbf{h}(\hat{\mathbf{x}}^-) \right) \tag{3.30}$$

$$P = (I - KH)P^- \tag{3.31}$$

3.5.3 State Structure and Attitude Handling

A typical state vector tracks position $\mathbf{p}$, velocity $\mathbf{v}$, attitude, and biases for both the gyroscope and accelerometer (along with a scale parameter if monocular vision is used). To avoid Euler-angle singularities, the attitude is handled via a 3-parameter error state around the current rotation matrix estimate $\hat{R}$. This approach is known as a Multiplicative Extended Kalman Filter (MEKF). Structurally, the EKF prediction-correction sequence shares the same architecture as non-linear state observers, with the key difference being that the correction gains are computed dynamically from the covariances.

CHAPTER

4

MOTION PLANNING

In the previous chapters we have studied how to *track* a given reference trajectory. In this chapter we study how to *generate* it: given a starting state, a goal, and a known static environment with obstacles, we want to compute a trajectory that is collision-free and that can actually be flown by the quadrotor, i.e. one that satisfies its dynamics and its actuation limits.

4.1 The Motion Planning Problem

Let $\mathcal{C}$ be the **configuration space** of the robot, the set of all the configurations $\mathbf{q}$ that completely describe the position of the robot in the workspace. Let $\mathcal{C}_{\text{free}} \subset \mathcal{C}$ be the set of the **free configurations**, i.e. those in which the robot does not intersect any obstacle.

Definition 4.1.1 : Geometric Motion Planning Problem

Given a start configuration $\mathbf{q}_s \in \mathcal{C}_{\text{free}}$ and a goal configuration $\mathbf{q}_g \in \mathcal{C}_{\text{free}}$, find a continuous path

$$\mathbf{q} : [0,1] \to \mathcal{C}_{\text{free}}, \quad \mathbf{q}(0) = \mathbf{q}_s, \ \mathbf{q}(1) = \mathbf{q}_g \tag{4.1}$$

or report failure if such a path does not exist.

If the environment is static and known, many planning algorithms can solve this *geometric* problem, the well known **piano movers' problem**. The problem is **purely geometric**: the robot is assumed to be able to move along *any* path of $\mathcal{C}_{\text{free}}$, without any constraint coming from its dynamics.

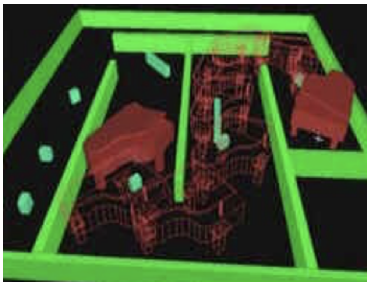


Figure 4.1: The piano movers' problem: the geometric planning problem in a known static environment.

For a flying robot the workspace is three-dimensional, with obstacles, and the planned path must be flown by a vehicle with its own dynamics.

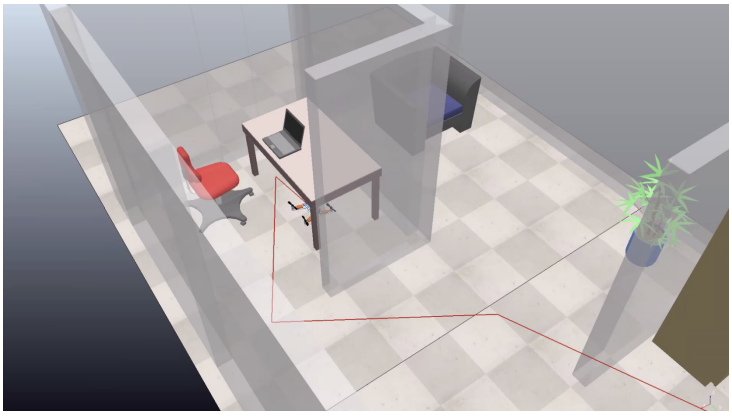


Figure 4.2: Motion planning for a flying robot: the planned path (in red) must be flown by a vehicle with its own dynamics.

4.2 What Is Special With Quadrotors

The quadrotor has two features that make the geometric formulation of Definition 4.1.1 insufficient.

4.2.1 Second Order Dynamics

The quadrotor is a **second order** dynamic system: the input acts at the acceleration level (the thrust T determines $\ddot{\mathbf{p}}$ and the torques determine $\ddot{\Theta}$, see Equation (2.60)). Consequences:

- A geometric path is **not** a motion: the feasibility of a path depends on the **time law** along it, i.e. on how fast the path is traveled (the accelerations, hence thrust and torques, depend on the time law).
- At a **corner** of the path (a point where the tangent is discontinuous) the velocity must vanish: flying through the corner at nonzero speed would require an instantaneous change of velocity, i.e. an infinite acceleration.
- Hence a corner is either followed by **stopping** there, or it is **smoothed**: but a smoothed path is a different path, that may collide with the obstacles.

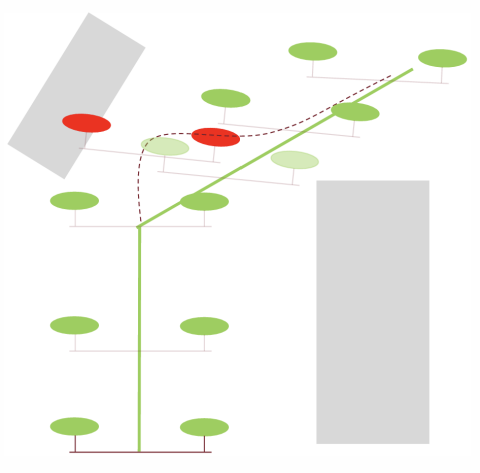


Figure 4.3: A path with a corner (in green): the vehicle must stop at the corner or follow a smoothed path (dashed), which can deviate from the original path and collide with the obstacles (in gray).

4.2.2 Underactuation

The quadrotor is **underactuated**: it has four inputs $\mathbf{u} = (T, \tau_\varphi, \tau_\vartheta, \tau_\psi)^\top$ and six degrees of freedom. Consequences:

- Not all the paths in the configuration space are feasible.
- The thrust is directed along the body axis $\mathbf{z}_b$, hence to accelerate horizontally the vehicle must **tilt**.

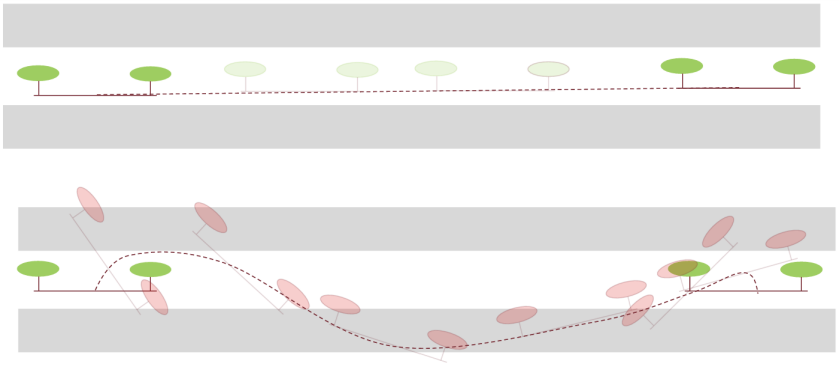


Figure 4.4: Top: a level translation along a corridor is a path in $\mathcal{C}$ but not a feasible motion. Bottom: a feasible motion tilts the vehicle to accelerate and to brake, and needs more room.

A level translation along a corridor (the vehicle keeps $\varphi = \vartheta = 0$) is a perfectly valid path in $\mathcal{C}$, but it is not a feasible motion, since it would require a horizontal acceleration with a vertical thrust. A feasible motion tilts the vehicle to accelerate and, symmetrically, to brake, hence it needs more room than the geometric path suggests.

4.2.3 Two Solution Approaches

There are two main ways to deal with these issues:

1. **Decoupled approach**: plan a collision-free geometric path, then optimize/approximate it locally to obtain a dynamically feasible, time-parametrized trajectory. It relies on **differential flatness**: a path of the flat outputs, made smooth enough and suitably timed, is a feasible trajectory.
2. **Kinodynamic planning**: explore directly the space of the feasible trajectories, by simulating the dynamics. Every branch of the search is feasible by construction.

We will study the first approach with a *visibility-based* roadmap planner, and the second one with the *RRT* (Rapidly-exploring Random Tree) algorithm, in both a random-input and a control-based version.

4.3 Flatness-Based Trajectory Generation

We have seen in Section 2.2.5 that the quadrotor is differentially flat, with flat outputs $\mathbf{y} = (x,\ y,\ z,\ \psi)^\top$. The consequence for planning is that **every sufficiently smooth curve $\mathbf{y}(t)$ is the output of a feasible trajectory**: the state and the inputs follow algebraically from $\mathbf{y}(t)$ and its derivatives, without integrating any differential equation. Planning is then reduced to choosing a curve in $\mathbb{R}^4$ that satisfies the boundary conditions, the collision avoidance and the input bounds.

In Section 2.2.5 we derived the map Ψ from the flat outputs to the inputs by using the angular velocity $\boldsymbol{\Omega}$. Here we derive the same map in terms of the roll-pitch-yaw angles, for the simplified model (Definition of the *Simplified Time Evolution of the Quadrotor*), in which $\ddot{\boldsymbol{\Theta}} = J^{-1}\boldsymbol{\Gamma}$. This form is the one used to evaluate a planned trajectory.

4.3.1 Attitude and Thrust From the Flat Outputs

Let $R_x(\varphi), R_y(\vartheta), R_z(\psi)$ be the elementary rotations about the axes $\mathbf{x}, \mathbf{y}, \mathbf{z}$, so that ${}^iR_b = R_z(\psi)R_y(\vartheta)R_x(\varphi)$. From the translational dynamics $\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\,{}^iR_b\mathbf{e}_3$ we get, by rotating the acceleration in the *heading frame* (the frame obtained by rotating SR_i about $\mathbf{z}$ by the yaw ψ):

$$R_z(\psi)^\top(\ddot{\mathbf{p}} - g\mathbf{e}_3) = -\frac{T}{m}R_y(\vartheta)R_x(\varphi)\mathbf{e}_3 = -\frac{T}{m}\begin{pmatrix}\sin\vartheta\cos\varphi\\ -\sin\varphi\\ \cos\vartheta\cos\varphi\end{pmatrix}. \tag{4.2}$$

Let us define the following three scalar quantities, that depend only on the flat outputs:

$$\nu_1 = \ddot{x}\cos\psi + \ddot{y}\sin\psi \tag{4.3}$$
$$\nu_2 = \ddot{x}\sin\psi - \ddot{y}\cos\psi \tag{4.4}$$
$$\gamma = \ddot{z} - g \tag{4.5}$$

so that the left-hand side of Equation (4.2) is $(\nu_1,\ -\nu_2,\ \gamma)^\top$. Solving for the attitude and the thrust:

$$\vartheta = \arctan\left(\frac{\nu_1}{\gamma}\right), \qquad \varphi = \arctan\left(\frac{\nu_2}{\gamma}\cos\vartheta\right) \tag{4.6}$$
$$T = m\sqrt{\ddot{x}^2 + \ddot{y}^2 + (\ddot{z} - g)^2} = m\|\ddot{\mathbf{p}} - g\mathbf{e}_3\| \tag{4.7}$$

(the expression of T is the one already found in Section 2.2.5). These expressions are valid for $T > 0$ and $|\varphi|, |\vartheta| < \frac{\pi}{2}$: in this region the third component of Equation (4.2) gives $\gamma = -\frac{T}{m}\cos\vartheta\cos\varphi < 0$, hence the denominator never vanishes. The values $\dot{x}, \dot{y}, \dot{z}, \dot{\psi}$ of the state are directly the derivatives of the flat outputs, while $\dot{\varphi}$ and $\dot{\vartheta}$ are obtained by differentiation.

4.3.2 Derivatives of the Angles

Both the angles in Equation (4.6) have the form $\alpha = \arctan\left(\frac{N}{\gamma}\right)$, with $\alpha = \vartheta$, $N = \nu_1$ for the pitch, and $\alpha = \varphi$, $N = \nu_2\cos\vartheta$ for the roll. Note that $\dot{\gamma} = z^{(3)}$ and $\ddot{\gamma} = z^{(4)}$. Differentiating $\tan\alpha = \frac{N}{\gamma}$ once and twice we obtain:

$$\dot{\alpha} = \cos^2\alpha\,\frac{\dot{N} - \tan\alpha\,\dot{\gamma}}{\gamma} \tag{4.8}$$
$$\ddot{\alpha} = -2\tan\alpha\,\dot{\alpha}^2 - 2\dot{\alpha}\frac{\dot{\gamma}}{\gamma} + \cos^2\alpha\,\frac{\ddot{N} - \tan\alpha\,\ddot{\gamma}}{\gamma}. \tag{4.9}$$

Pitch: $N = \nu_1$, and by differentiating (using $\dot{\nu}_1$, $\ddot{\nu}_1$ as defined below):

$$\dot{\nu}_1 = \left(x^{(3)} + \ddot{y}\dot{\psi}\right)\cos\psi + \left(y^{(3)} - \ddot{x}\dot{\psi}\right)\sin\psi \tag{4.10}$$

$$\ddot{\nu}_1 = \left(x^{(4)} + 2y^{(3)}\dot{\psi} + \ddot{y}\ddot{\psi} - \ddot{x}\dot{\psi}^2\right)\cos\psi + \left(y^{(4)} - 2x^{(3)}\dot{\psi} - \ddot{x}\ddot{\psi} - \ddot{y}\dot{\psi}^2\right)\sin\psi. \tag{4.11}$$

For example, by using $\cos^2\vartheta = \frac{1}{1+\tan^2\vartheta}$ in Equation (4.8):

$$\dot{\vartheta} = \frac{1}{1+\tan^2\vartheta}\frac{1}{\ddot{z}-g}\left[\left(x^{(3)} + \ddot{y}\dot{\psi}\right)\cos\psi + \left(y^{(3)} - \ddot{x}\dot{\psi}\right)\sin\psi - z^{(3)}\tan\vartheta\right]. \tag{4.12}$$

Roll: $N = \nu_2\cos\vartheta$, with:

$$\dot{\nu}_2 = \left(x^{(3)} + \ddot{y}\dot{\psi}\right)\sin\psi - \left(y^{(3)} - \ddot{x}\dot{\psi}\right)\cos\psi \tag{4.13}$$

$$\ddot{\nu}_2 = \left(x^{(4)} + 2y^{(3)}\dot{\psi} + \ddot{y}\ddot{\psi} - \ddot{x}\dot{\psi}^2\right)\sin\psi - \left(y^{(4)} - 2x^{(3)}\dot{\psi} - \ddot{x}\ddot{\psi} - \ddot{y}\dot{\psi}^2\right)\cos\psi \tag{4.14}$$

and, by the product rule:

$$\dot{N} = \cos\vartheta\,\dot{\nu}_2 - \sin\vartheta\,\dot{\vartheta}\,\nu_2 \tag{4.15}$$

$$\ddot{N} = \cos\vartheta\,\ddot{\nu}_2 - 2\sin\vartheta\,\dot{\vartheta}\,\dot{\nu}_2 - \left(\cos\vartheta\,\dot{\vartheta}^2 + \sin\vartheta\,\ddot{\vartheta}\right)\nu_2. \tag{4.16}$$

For example:

$$\dot{\varphi} = \frac{\cos^2\varphi}{\ddot{z}-g}\left[\cos\vartheta\,\dot{\nu}_2 - \sin\vartheta\,\dot{\vartheta}\,\nu_2 - z^{(3)}\tan\varphi\right]. \tag{4.17}$$

The pitch must be computed first (angle, first and second derivative), since the roll depends on $\vartheta, \dot{\vartheta}, \ddot{\vartheta}$. Note that $\dot{\varphi}, \dot{\vartheta}$ need the jerk $\mathbf{p}^{(3)}$ (and $\dot{\psi}$), while $\ddot{\varphi}, \ddot{\vartheta}$ need the snap $\mathbf{p}^{(4)}$ (and $\ddot{\psi}$).

4.3.3 Inputs From the Flat Outputs

The inputs are finally given by:

$$T = m\|\ddot{\mathbf{p}} - g\mathbf{e}_3\|, \qquad \tau_\varphi = I_x\ddot{\varphi}, \quad \tau_\vartheta = I_y\ddot{\vartheta}, \quad \tau_\psi = I_z\ddot{\psi}. \tag{4.18}$$

The flat outputs derivatives needed to compute each quantity are:

- T, φ, ϑ: the acceleration $\ddot{\mathbf{p}}$ (and ψ);
- $\dot{\varphi}, \dot{\vartheta}$: the jerk $\mathbf{p}^{(3)}$ (and $\dot{\psi}$);
- the torques $\tau_\varphi, \tau_\vartheta, \tau_\psi$: the snap $\mathbf{p}^{(4)}$ (and $\ddot{\psi}$).

These are the same flat outputs, and the same derivatives, of the dynamic feedback linearization, in which the relative degrees of (x, y, z, ψ) are $(4, 4, 4, 2)$ after the extension of the thrust with $T, \dot{T}$: the DFL controller tracks exactly the trajectories generated in this way.

Observation 4.3.1

For the *complete* model, the torques are given by $\boldsymbol{\Gamma} = J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega}\times J\boldsymbol{\Omega}$, with $\boldsymbol{\Omega} = M(\boldsymbol{\Theta})\dot{\boldsymbol{\Theta}}$ (the matrix relating the angular velocity to the Euler angles rates, see the table of the DFL), $\dot{\boldsymbol{\Omega}} = M(\boldsymbol{\Theta})\ddot{\boldsymbol{\Theta}} + \dot{M}(\boldsymbol{\Theta})\dot{\boldsymbol{\Theta}}$. Since $\boldsymbol{\Theta}, \dot{\boldsymbol{\Theta}}, \ddot{\boldsymbol{\Theta}}$ are functions of the flat outputs (and their derivatives up to the snap), the torques are still a function of the flat outputs: the complete model is flat as well, with the same flat outputs.

4.3.4 Feasibility of 3D Trajectories

We can summarize the conditions under which a trajectory $\mathbf{y}(t)$ is feasible for the quadrotor:

- The robot is able to track any given trajectory of the 3D position and of the yaw angle, provided that it is "smooth enough".
- To guarantee the **continuity of the inputs**, the position trajectory must be continuous up to the fourth derivative ($\mathbf{p} \in C^4$, since the torques depend on the snap) and the yaw trajectory up to the second derivative ($\psi \in C^2$).
- To satisfy the **actuation constraints**, the total duration of the trajectory has to be "large enough": this is achieved with the time scaling, discussed later.
- The near-hovering model also requires moderate roll and pitch angles.

4.3.5 Paths With Corners

A piecewise-linear path $\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n$ (with $\mathbf{p}_i \in \mathbb{R}^3$ the *nodes* of the path) is feasible if the robot **stops at every node**. On the segment from $\mathbf{p}_i$ to $\mathbf{p}_{i+1}$, traveled in the time interval $[t_i, t_i + \Delta t_i]$ (with $\Delta t_i > 0$ the duration of the segment):

$$\mathbf{p}(t) = \mathbf{p}_i + \sigma\left(\frac{t - t_i}{\Delta t_i}\right)(\mathbf{p}_{i+1} - \mathbf{p}_i), \quad t \in [t_i, t_i + \Delta t_i] \tag{4.19}$$

where $\sigma : [0,1] \to [0,1]$ is the polynomial of degree 9

$$\sigma(s) = 126s^5 - 420s^6 + 540s^7 - 315s^8 + 70s^9. \tag{4.20}$$

It holds $\sigma(0) = 0, \sigma(1) = 1$ and

$$\sigma'(s) = 630\, s^4(1-s)^4 \tag{4.21}$$

(the prime is the derivative with respect to s), hence the velocity, the acceleration, the jerk and the snap vanish at both the ends of each segment, so that $\mathbf{p} \in C^4$ also at the corners. During the motion, the thrust stays in the vertical plane containing the segment: the vehicle tilts forward to accelerate and backward to brake.

This solution is feasible, but **slow**, since the robot stops at every node. A smooth curve that goes through (or near) the nodes avoids the stops: this is the motivation of the spline interpolation seen below.

4.4 Visibility-Based Motion Planning

The first approach is a *decoupled* one, based on flatness. It is composed of four steps:

1. creation of a **visibility-based probabilistic roadmap** in the space of the flat outputs;
2. **search** of a piecewise-linear collision-free path by using the roadmap;
3. approximation of the computed path through a smooth **spline**;
4. **time scaling**: choice of the duration of the trajectory such that the input bounds are satisfied.

4.4.1 Visibility Roadmap Construction

The goal is to build a roadmap with *few nodes* but sufficient for solving the global planning problem. Let $L(\mathbf{q}, \mathbf{q}')$ be a **local method**, a procedure that connects two configurations with a path (for example the straight line segment). The set of the configurations that can be reached from $\mathbf{q}$ through L is its visibility domain.

Definition 4.4.1 : Visibility Domain, Guard, Coverage

Given a local method L, the **visibility domain** of $\mathbf{q}$ is

$$\mathcal{V}_L(\mathbf{q}) = \left\{ \mathbf{q}' \in \mathcal{C}_{\text{free}} \text{ such that } L(\mathbf{q}, \mathbf{q}') \subset \mathcal{C}_{\text{free}} \right\}. \tag{4.22}$$

The configuration $\mathbf{q}$ is called the **guard** of $\mathcal{V}_L(\mathbf{q})$. A set of guards is a **coverage** of $\mathcal{C}_{\text{free}}$ if the union of their visibility domains is $\mathcal{C}_{\text{free}}$.

Note that a finite coverage may not always exist: it depends on both the shape of $\mathcal{C}_{\text{free}}$ and on the local method L. The roadmap is a graph, whose nodes are collision-free configurations and whose edges are the paths computed by L. At each iteration a random collision-free sample $\mathbf{q}$ is generated, and it is added to the roadmap if and only if one of the following conditions is satisfied:

- $\mathbf{q}$ does not belong to the visibility domain of any guard of the current roadmap. In this case $\mathbf{q}$ becomes a new **guard**: it enlarges the coverage of $\mathcal{C}_{\text{free}}$ (and it starts a new connected component of the roadmap).
- $\mathbf{q}$ lies in the intersection of the visibility domains of at least two guards belonging to *different connected components* of the current roadmap. In this case $\mathbf{q}$ becomes a **connector**, and it is connected to those guards: it enhances the connectivity of the roadmap.

Otherwise the sample is **discarded**, so that the roadmap stays small.

Algorithm 2 Visibility Roadmap Construction

```
1:  G ← empty graph (nodes are guards and connectors)
2:  repeat
3:      q ← random sample of C_free
4:      S ← set of the guards q_g of G such that q ∈ V_L(q_g)
5:      if S = ∅ then
6:          add q to G as a guard
7:      else if S contains guards of at least two different connected components of G then
8:          add q to G as a connector, with an edge to one guard of S for each of those components
9:      else
10:         discard q
11:     end if
12: until the stopping criterion is met
```

The stopping criterion is chosen by the designer, for example a maximum number of samples or a number of consecutive rejected samples. The connections are used to enhance the connectivity, but they are also useful to increase the coverage.

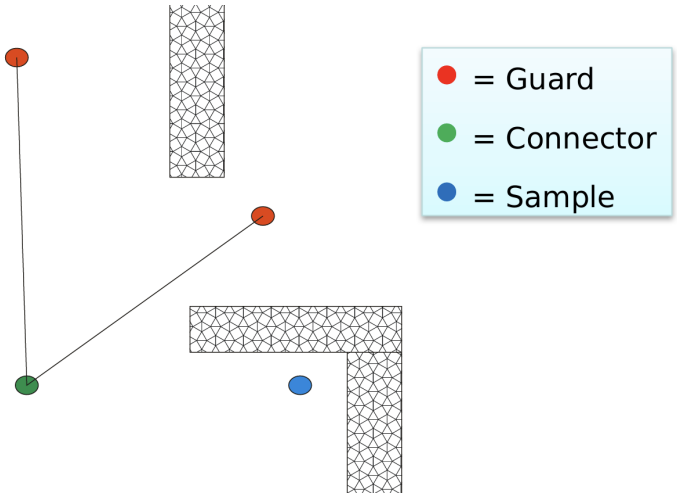


Figure 4.5: Visibility roadmap construction: guards (red), connector (green) and a sample (blue) that is not visible from any guard. In the original algorithm it would become a new guard; if the visibility domains of the connectors are counted too, it is discarded.

The blue sample in Figure 4.5 is not visible from any guard, hence with the rule above it would become a new guard; if the visibility domains of the connectors are counted too in the coverage, it is discarded.

Roadmap for Quadrotors

For the quadrotor:

- **Planning space**: the space of the flat outputs $\mathbf{y} = (x, y, z, \psi)^\top$.
- **Collision checking by obstacle expansion**: the obstacles are grown by the radius r_v of a sphere, centered at the center of mass, that contains the vehicle (r_v is the arm length l plus the propeller radius; in the example below $l = 0.25$ m). Then only the position of the center of mass has to be checked against the grown obstacles, and this is valid for *any* attitude of the vehicle.
- The yaw ψ is actuated independently and does not change the sphere containing the vehicle: it can be neglected, and the roadmap is built in $\mathbb{R}^3$.
- **Local method**: straight lines.

4.4.2 Search for a Safe Path

The goal of this step is to compute a collision-free piecewise-linear path from the starting position to the goal, or to return failure if such a path does not exist. The search is made iteratively:

1. try to connect, by using straight lines, the start $\mathbf{q}_s$ and the goal $\mathbf{q}_g$ to two roadmap nodes $\mathbf{q}_a, \mathbf{q}_b$ that belong to the *same* connected component of the roadmap;
2. if this succeeds, a solution exists and a graph search (A*) on the roadmap gives the path, with the length of the segments as cost;
3. if not, the roadmap is expanded (more samples are generated) and the connection is tried again.

The obtained piecewise-linear path is collision-free, but it is **not dynamically feasible**, since it has corners.

4.4.3 Spline Interpolation

The next step computes a smooth solution that approximates the piecewise-linear path, by using a spline that interpolates its nodes. This spline may diverge significantly from the safe piecewise-linear path and, consequently, become unsafe. The solution is to introduce a **via point** in the middle of any unsafe stretch of the path: it pulls the spline back toward the segment. The procedure is repeated until the whole spline is collision-free.

The geometric aspect and the temporal one are separated. Let $\lambda \in [0,1]$ be a **non-dimensional time** (we do not use τ, that denotes the torques), and let

$$\mathbf{y}_i = (x_i,\ y_i,\ z_i,\ \psi_i)^\top, \quad i = 1, \dots, n \tag{4.23}$$

be the nodes of the path (the via points). We look for the splines

$$s_x(\lambda),\ s_y(\lambda),\ s_z(\lambda) \in C^4([0,1]; \mathbb{R}) \quad \text{of order 6 (degree 5)} \tag{4.24}$$

$$s_\psi(\lambda) \in C^2([0,1]; \mathbb{R}) \quad \text{of order 4 (cubic)} \tag{4.25}$$

(the order of a polynomial spline is its degree plus one) such that

$$s_x(\lambda_i) = x_i, \quad s_y(\lambda_i) = y_i, \quad s_z(\lambda_i) = z_i, \quad s_\psi(\lambda_i) = \psi_i, \quad i = 1, \dots, n \tag{4.26}$$

where $0 = \lambda_1 < \lambda_2 < \cdots < \lambda_n = 1$ are the **knots**. We define $\mathbf{s}_\mathbf{p}(\lambda) = (s_x, s_y, s_z)^\top$ and $\mathbf{s} = (\mathbf{s}_\mathbf{p}^\top, s_\psi)^\top$. The orders are chosen according to the smoothness required for the flat outputs: order 6 for the position gives a continuous snap, order 4 for the yaw gives a continuous yaw acceleration.

About the Notation 4.4.1

For the splines, the superscript in round brackets denotes the derivative with respect to the non-dimensional time λ, e.g. $\mathbf{s}_\mathbf{p}^{(4)} = \frac{d^4 \mathbf{s}_\mathbf{p}}{d\lambda^4}$, while the dot denotes, as usual, the derivative with respect to the real time t.

The interpolation conditions are not sufficient to determine the splines: a piecewise polynomial of degree 5 on $n-1$ intervals has $6(n-1)$ coefficients, while the C^4 continuity at the $n-2$ internal knots gives $5(n-2)$ conditions and the interpolation of the n values gives n conditions, so that 4 conditions are missing (2 at each end); analogously, 2 conditions are missing for the cubic yaw spline (1 at each end). They are the **end conditions**, for example rest at the start and at the goal: $\mathbf{s}_\mathbf{p}^{(1)} = \mathbf{s}_\mathbf{p}^{(2)} = \mathbf{0}$ at $\lambda = 0$ and $\lambda = 1$ (and $s_\psi^{(1)} = 0$ for the yaw).

Choice of the Knots

The knots $(\lambda_1, \dots, \lambda_n)$ determine how the (non-dimensional) time is distributed among the stretches of the path, and they are chosen by solving an optimization problem. Let

$$\Delta\lambda_i = \lambda_{i+1} - \lambda_i, \quad i = 1, \dots, n-1 \tag{4.27}$$

be the optimization variables, subject to the constraints

$$0 < \Delta\lambda_i < 1, \quad i = 1, \dots, n-1, \qquad \sum_{i=1}^{n-1} \Delta\lambda_i = 1. \tag{4.28}$$

The objective function is

$$\int_0^1 \left[a \left\| \mathbf{s}_\mathbf{p}^{(4)}(\lambda) \right\|^2 + b \left(s_\psi^{(2)}(\lambda) \right)^2 \right] d\lambda \tag{4.29}$$

with $a, b > 0$ weights on the translational and on the yaw part. This is the **minimum snap** criterion: the knots minimize the (weighted) integral of the squared snap and yaw acceleration, since the snap is the quantity from which the torques are computed. A simple alternative, used for comparison below, is the **chordal** choice of the knots, in which $\Delta\lambda_i$ is proportional to the length $\|\mathbf{p}_{i+1} - \mathbf{p}_i\|$ of the i-th segment.

4.4.4 Time Scaling

The splines give a path $\mathbf{s}(\lambda)$, with no real time yet. We now choose the time needed to complete the trajectory, in order to satisfy the input bounds. If the time to navigate through the nodes is changed by a factor $k > 0$, i.e. $t_i = k\lambda_i$, the result is simply a **time-scaled version** of the non-dimensional solution:

$$\mathbf{p}(t) = \mathbf{s}_\mathbf{p}\left(\frac{t}{k}\right), \quad \psi(t) = s_\psi\left(\frac{t}{k}\right), \quad t \in [0, k]. \tag{4.30}$$

The geometric path is unchanged (hence it is still collision-free), and the derivatives scale as follows:

$$\dot{\mathbf{p}} = \frac{\mathbf{s}_\mathbf{p}^{(1)}}{k}, \quad \ddot{\mathbf{p}} = \frac{\mathbf{s}_\mathbf{p}^{(2)}}{k^2}, \quad \mathbf{p}^{(3)} = \frac{\mathbf{s}_\mathbf{p}^{(3)}}{k^3}, \quad \mathbf{p}^{(4)} = \frac{\mathbf{s}_\mathbf{p}^{(4)}}{k^4}, \quad \ddot{\psi} = \frac{s_\psi^{(2)}}{k^2} \tag{4.31}$$

where the derivatives of $\mathbf{s}_\mathbf{p}$ and s_ψ are evaluated at $\lambda = t/k$. The total duration of the trajectory is k. The factor k is chosen by solving an optimization problem:

- **variable**: k;
- **constraints**: along the whole trajectory, $T \le T_M$, $|\tau_\varphi| \le \tau_{\varphi,M}$, $|\tau_\vartheta| \le \tau_{\vartheta,M}$, $|\tau_\psi| \le \tau_{\psi,M}$, where T_M is the maximum thrust and $\tau_{\varphi,M}$, $\tau_{\vartheta,M}$, $\tau_{\psi,M}$ are the maximum torques that the actuators can produce;
- **objective function**: $\min k$ (the total time).

The inputs along the trajectory are computed from the flat outputs as seen in the previous section. As $k \to \infty$ all the derivatives of the position vanish, hence $T = m\|\ddot{\mathbf{p}} - g\mathbf{e}_3\| \to mg$ and the torques tend to zero. Therefore a feasible k exists if $mg < T_M$ (the vehicle must be able to hover). The minimum feasible k can be found with a one-dimensional search, e.g. a bisection, under the assumption (verified in the example below) that the peak values of the inputs decrease as k grows.

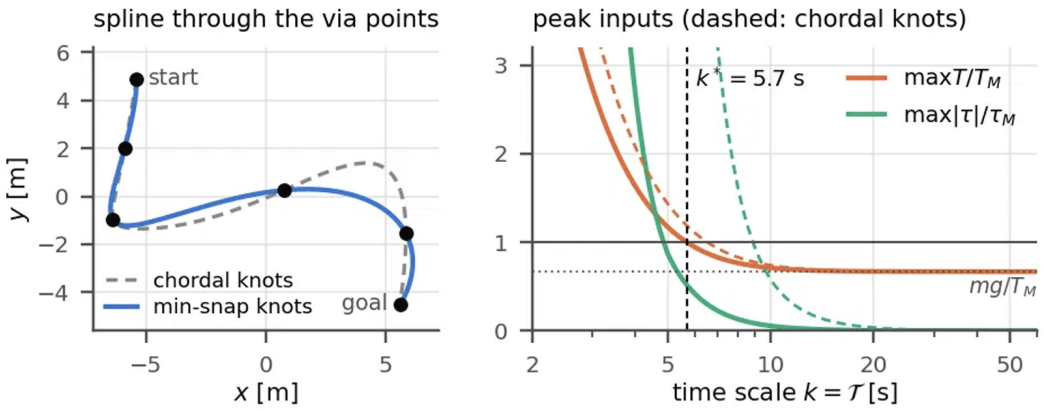


Figure 4.6: Time scaling for the via points of the example below (horizontal projection, constant height, $\psi = 0$). Left: the splines through the via points, with min-snap knots (blue) and chordal knots (dashed gray). Right: peak values of the thrust, normalized with T_M, and of the torques, normalized with τ_M, as functions of the time scale k; the dashed curves are relative to the chordal knots. With the min-snap knots the optimum is $k^* = 5.7$ s, where the thrust bound is active.

Illustration: for the via points of the example of the next section, with constant height, $\psi = 0$, $m = 0.5$ kg, $I_x = I_y = 4.9 \cdot 10^{-3}$ kg m^2, $T_M = 1.5\,mg$ and $\tau_M = 0.05$ N m (the bound on each torque), we get:

- with the min-snap knots, the integral of the squared snap is about 20 times smaller than with the chordal knots, and the optimal time scale is $k^* = 5.7$ s instead of 8.9 s;
- at k^* the thrust bound is active, while the torques are at 52% of their bound, with a tilt up to 40°.

4.4.5 Choice of the Yaw

Since ψ is actuated independently, it can be neglected during the roadmap construction. An alternative to a given yaw is to choose

$$\psi = \operatorname{atan2}(\dot{y}, \dot{x}) \tag{4.32}$$

so that the robot **points in the direction of motion**. With $\rho = \dot{x}^2 + \dot{y}^2$ the squared horizontal speed, the yaw rate and acceleration are:

$$\dot{\psi} = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\rho} \tag{4.33}$$

$$\ddot{\psi} = \frac{\dot{x}\, y^{(3)} - \dot{y}\, x^{(3)}}{\rho} - 2\dot{\psi}\frac{\dot{x}\ddot{x} + \dot{y}\ddot{y}}{\rho}. \tag{4.34}$$

This choice is defined only when the horizontal speed is nonzero: it cannot be used at rest (start and goal) nor in vertical flight. Moreover, $\psi \in C^2$ requires $x, y \in C^3$, which is already guaranteed by the position spline (C^4).

4.4.6 Example

We consider an environment of $7 \times 7 \times 3$ m with walls: the long walls have openings, where the path can cross them.

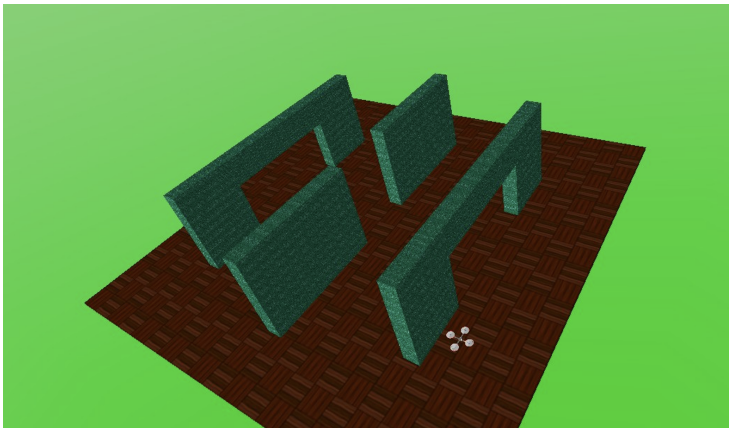


Figure 4.7: Example environment, $7 \times 7 \times 3$ m, with walls; the long walls have openings where the path can cross them.

The roadmap, in the horizontal projection, is shown in Figure 4.8: the obstacles (in black) are grown by the arm length (in cyan). The roadmap has few nodes (guards and connectors), and the start and the goal are connected to it.

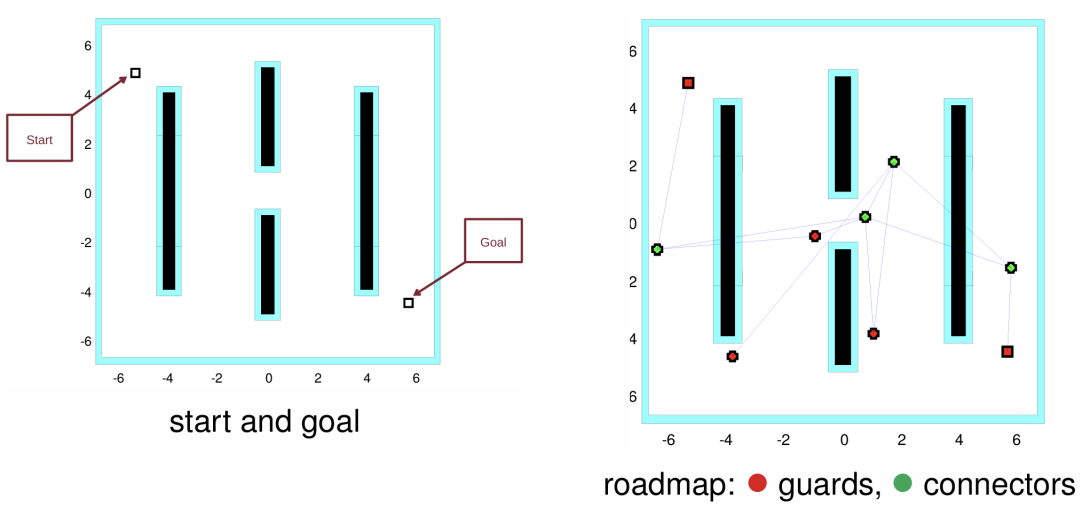


Figure 4.8: Start, goal and visibility roadmap in the horizontal projection: obstacles in black, obstacles grown by the arm length in cyan, guards in red and connectors in green.

The path computed by A* on the roadmap is piecewise linear. The spline through its nodes leaves the safe path in the circled stretch of Figure 4.9, hence a via point is introduced in the middle of the unsafe stretch, pulling the spline back toward the segment; the procedure is repeated until the whole spline is collision-free. Then the knots are optimized and the time is scaled.

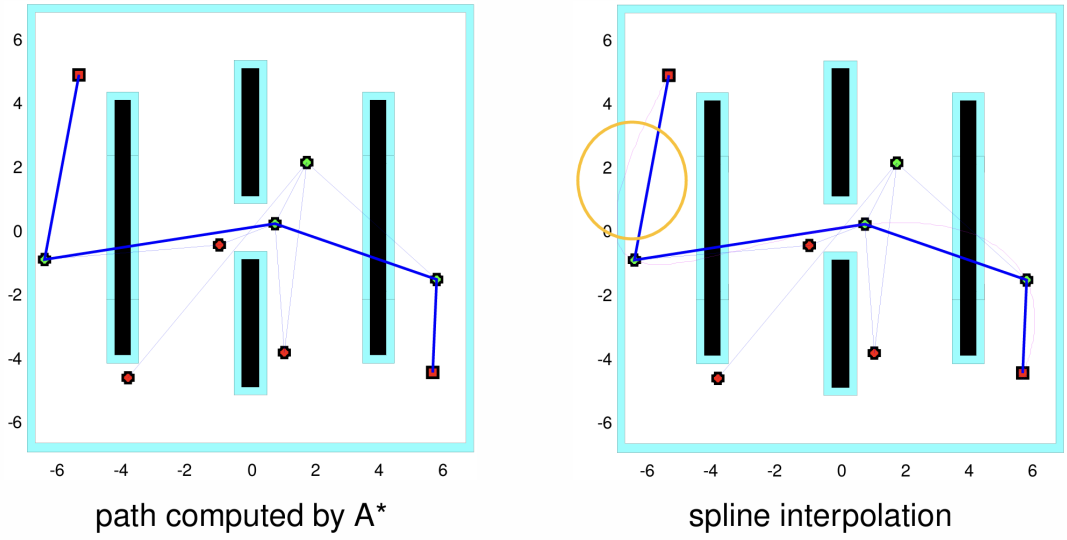


Figure 4.9: Path computed by A* (left) and spline interpolation of its nodes (right): the spline leaves the safe piecewise-linear path in the circled stretch.

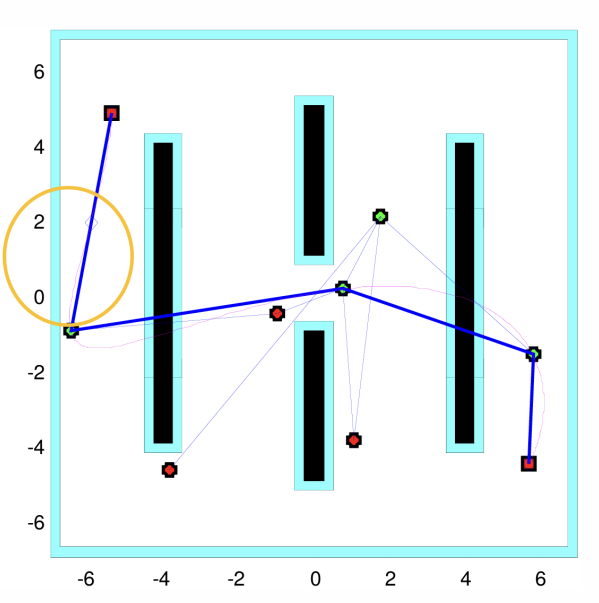


Figure 4.10: Introduction of an additional via point in the middle of the unsafe stretch: the spline is pulled back toward the segment.

Since the spatial shape of the spline depends on the knots, it is prudent to check again the collision-freeness of the spline after the optimization of the knots, and to insert further via points if needed.

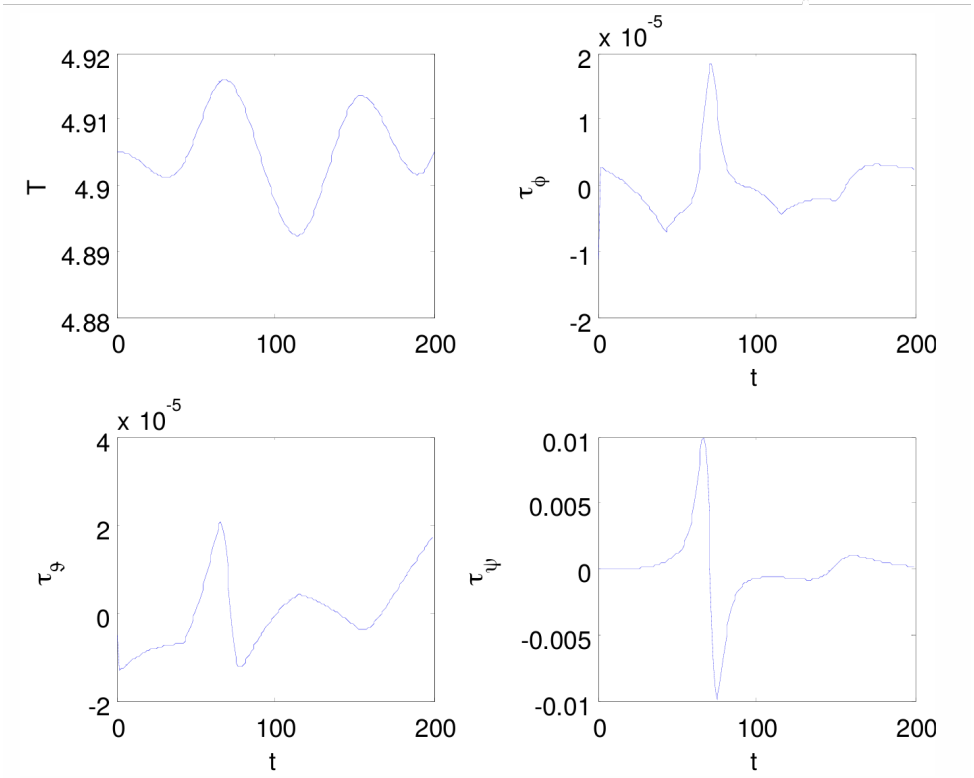


Figure 4.11: Control inputs $T, \tau_\varphi, \tau_\vartheta, \tau_\psi$ along the trajectory after time scaling.

Figure 4.11 shows the control inputs after the time scaling: the thrust T is close to mg and the torques are very small, i.e. the trajectory is smooth and slow. The inputs are continuous, as guaranteed by $\mathbf{p} \in C^4$ and $\psi \in C^2$.

4.4.7 Remarks on the Visibility-Based Approach

- The visibility algorithm is suitable for finding a roadmap with **few nodes**, easy to handle but possibly not containing the shortest path. A **smart cutting** of the path can reduce its length: it replaces a portion of the path with a shortcut between two points of the path that see each other.
- The trajectory would be even smoother if the spline passed *near* the nodes of the roadmap instead of interpolating them: the optimizer would have more freedom.
- The prior geometric path restricts the generated trajectory to a given **homotopy class**, which may not contain a globally optimal (or even feasible) trajectory.

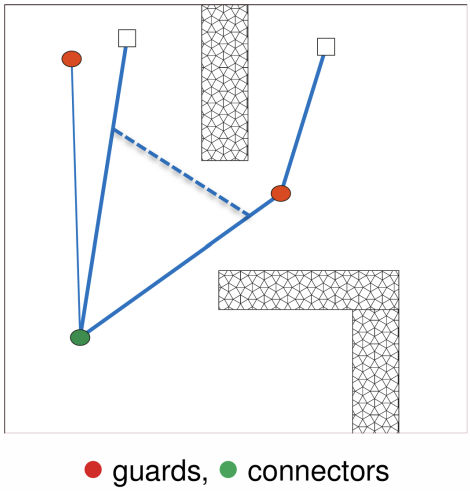


Figure 4.12: Smart cutting: the roadmap may not contain the shortest path (solid), a shortcut (dashed) between two points of the path that see each other makes it shorter.

4.5 RRT-Based Motion Planning

The second approach is the **kinodynamic** one: a search tree is incrementally grown in the *state space* by applying control inputs over short time intervals, in order to reach new nodes. The main features of the RRT are:

- the exploration is biased toward the unexplored portions of the space (*rapidly-exploring*);
- it is suitable for high-dimensional spaces;
- it takes into account both kinematic and dynamic constraints;
- it generates a trajectory directly in the state space, and provides the control inputs needed to execute it.

Here the planning space is the state space of the quadrotor, with state $\boldsymbol{\xi} \in \mathbb{R}^{12}$ and dynamics $\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u}$ as in Equation (1.19).

4.5.1 The Algorithm

Let $\rho(\boldsymbol{\xi}, \boldsymbol{\xi}')$ be a **metric** on the state space (for example the Euclidean distance), and $\Delta t > 0$ a fixed integration time. At each iteration:

1. extract a random state $\boldsymbol{\xi}_{\text{rand}}$;
2. select the node $\boldsymbol{\xi}_{\text{near}}$ of the tree that is the closest to $\boldsymbol{\xi}_{\text{rand}}$ according to the metric ρ;

3. compute an input $\mathbf{u}$ that steers the robot toward $\boldsymbol{\xi}_{\text{rand}}$;
4. apply $\mathbf{u}$ for a time Δt starting from $\boldsymbol{\xi}_{\text{near}}$, reaching the state $\boldsymbol{\xi}_{\text{new}}$;
5. if the path from $\boldsymbol{\xi}_{\text{near}}$ to $\boldsymbol{\xi}_{\text{new}}$ is **safe** (collision-free, with the obstacles grown as seen above), add $\boldsymbol{\xi}_{\text{new}}$ to the tree and save $\mathbf{u}$; otherwise discard it.

Algorithm 3 Kinodynamic RRT

Require: $\boldsymbol{\xi}_{\text{start}}$, $\boldsymbol{\xi}_{\text{goal}}$, Δt, tolerance $\varepsilon_g > 0$, maximum number of iterations $N_{\max}$

$\mathcal{T} \leftarrow$ tree with the only node $\boldsymbol{\xi}_{\text{start}}$
for $j = 1, \ldots, N_{\max}$ **do**
  $\boldsymbol{\xi}_{\text{rand}} \leftarrow \textsc{RandomState}(j)$
  $\boldsymbol{\xi}_{\text{near}} \leftarrow$ node of $\mathcal{T}$ minimizing $\rho(\boldsymbol{\xi}, \boldsymbol{\xi}_{\text{rand}})$
  $(\mathbf{u}, \boldsymbol{\xi}_{\text{new}}) \leftarrow \textsc{Steer}(\boldsymbol{\xi}_{\text{near}}, \boldsymbol{\xi}_{\text{rand}}, \Delta t)$
  if the path from $\boldsymbol{\xi}_{\text{near}}$ to $\boldsymbol{\xi}_{\text{new}}$ is safe **then**
    add $\boldsymbol{\xi}_{\text{new}}$ to $\mathcal{T}$ as a child of $\boldsymbol{\xi}_{\text{near}}$, saving $\mathbf{u}$
    if $\|\boldsymbol{\xi}_{\text{new}} - \boldsymbol{\xi}_{\text{goal}}\| < \varepsilon_g$ **then**
      return the path from $\boldsymbol{\xi}_{\text{start}}$ to $\boldsymbol{\xi}_{\text{new}}$ in $\mathcal{T}$, with the saved inputs
    end if
  end if
end for
return failure

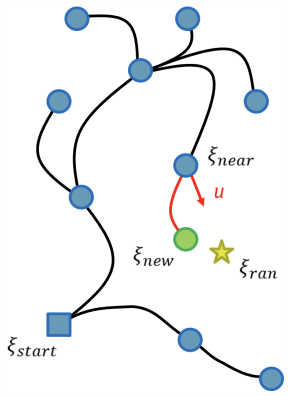


Figure 4.13: One iteration of the RRT: from $\boldsymbol{\xi}_{\text{near}}$ the input $\mathbf{u}$ steers the robot toward $\boldsymbol{\xi}_{\text{rand}}$, reaching $\boldsymbol{\xi}_{\text{new}}$.

Trajectory Reconstruction

Once $\|\boldsymbol{\xi}_{\text{last}} - \boldsymbol{\xi}_{\text{goal}}\| < \varepsilon_g$, where $\boldsymbol{\xi}_{\text{last}}$ is the last node added, the algorithm stops. The solution trajectory is found by traversing the tree backwards from $\boldsymbol{\xi}_{\text{last}}$ to $\boldsymbol{\xi}_{\text{start}}$, together with the saved inputs. The last stretch, from $\boldsymbol{\xi}_{\text{last}}$ to $\boldsymbol{\xi}_{\text{goal}}$, might be computed (if necessary) by using any local planner.

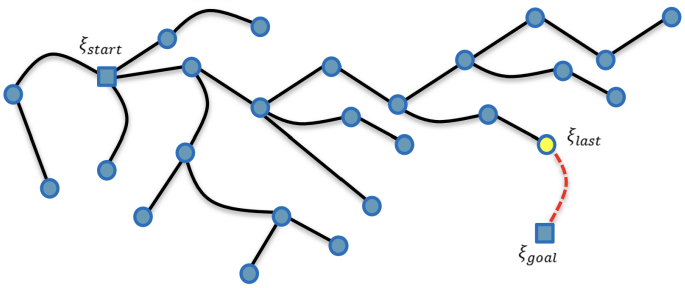


Figure 4.14: Trajectory reconstruction: the path is found by traversing the tree backwards from $\boldsymbol{\xi}_{\text{last}}$ to $\boldsymbol{\xi}_{\text{start}}$; the last stretch (dashed) to $\boldsymbol{\xi}_{\text{goal}}$ is computed with a local planner.

4.5.2 Exploration vs. Exploitation

The tree will eventually cover the connected component of $\mathcal{C}_{\text{free}}$ that contains the start, coming arbitrarily close to any goal belonging to the same component: this property is called **probabilistic completeness** (the probability of finding a solution, if it exists, tends to 1 as the number of iterations tends to infinity). However, the convergence is often slow. A common remedy is to switch between two phases:

- **exploration**: the tree is expanded toward a random state;
- **exploitation**: the tree is expanded toward the goal, i.e. $\boldsymbol{\xi}_{\text{rand}} = \boldsymbol{\xi}_{\text{goal}}$.

The choice is made with an ϵ-greedy strategy: at the iteration j, let $c \in [0,1]$ be a random number (uniformly distributed) and let

$$\epsilon_j = \frac{\epsilon_0}{1 + \gamma_\epsilon \, j} \tag{4.35}$$

with $\epsilon_0 \in (0,1]$ the initial exploration threshold and $\gamma_\epsilon > 0$ a decay rate. Then

$$\begin{cases} c < \epsilon_j & \Longrightarrow \text{ exploration} \\ c \geq \epsilon_j & \Longrightarrow \text{ exploitation.} \end{cases} \tag{4.36}$$

Exploration dominates at the beginning, exploitation as the tree grows. Note that ϵ_j is a threshold on the random number c, not to be confused with the goal tolerance ε_g.

4.5.3 Steering With Random Inputs

In the simplest version of the STEER function the input is chosen among random candidates. At each iteration:

1. extract 10 random input vectors: the angular velocities $\omega_1, \ldots, \omega_4$ of the four motors are random values between 0 and 1000 rad/s;
2. compute the corresponding thrust and torques by using the transformation seen in the modeling section, with $f_i = b\omega_i^2$ and $\tau_{R,i} = d\omega_i^2$:

$$T = b\left(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2\right) \tag{4.37}$$

$$\tau_\varphi = lb\left(\omega_2^2 - \omega_4^2\right), \quad \tau_\vartheta = lb\left(\omega_1^2 - \omega_3^2\right) \tag{4.38}$$

$$\tau_\psi = d\left(-\omega_1^2 + \omega_2^2 - \omega_3^2 + \omega_4^2\right); \tag{4.39}$$

3. integrate the equations of motion (with a numerical integrator, e.g. `ode45` in MATLAB) for the constant time $\Delta t = 0.1$ s;
4. choose the input vector that brings the robot *safely* closer to $\boldsymbol{\xi}_{\text{rand}}$.

The simulation results are poor: the tree spreads slowly and erratically, and after hours of computation the goal is still not reached.

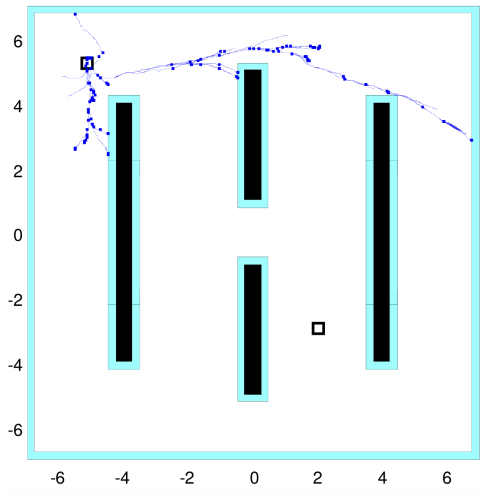


Figure 4.15: RRT with random inputs in the example environment: after 6 hours the tree has spread slowly and erratically, and the goal (square) is still not reached.

4.5.4 Control-Based Steering

A better solution is to combine the RRT with a **controller** that ensures that the quadrotor moves toward $\boldsymbol{\xi}_{\text{rand}}$. For example, a **variable LQR** controller is obtained by linearizing the dynamics around the current node $\boldsymbol{\xi}_{\text{near}}$ and the hovering input $\mathbf{u}_h = (mg,\ 0,\ 0,\ 0)^\top$:

$$A = \frac{\partial \dot{\boldsymbol{\xi}}}{\partial \boldsymbol{\xi}}\Big|_{\substack{\mathbf{u}=\mathbf{u}_h \\ \boldsymbol{\xi}=\boldsymbol{\xi}_{\text{near}}}}, \qquad B = \frac{\partial \dot{\boldsymbol{\xi}}}{\partial \mathbf{u}}\Big|_{\substack{\mathbf{u}=\mathbf{u}_h \\ \boldsymbol{\xi}=\boldsymbol{\xi}_{\text{near}}}} \tag{4.40}$$

where $\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u}$. The gain matrix K is computed (e.g. with the `lqr` function of MATLAB) as the one minimizing the cost

$$\int_0^\infty \left(\mathbf{e}^\top Q_c\, \mathbf{e} + \tilde{\mathbf{u}}^\top R_c\, \tilde{\mathbf{u}}\right) dt \tag{4.41}$$

for the linear system $\dot{\mathbf{e}} = A\mathbf{e} + B\tilde{\mathbf{u}}$, where $\mathbf{e}$ is the state error and $\tilde{\mathbf{u}}$ the deviation of the input from $\mathbf{u}_h$, with $Q_c \succeq 0$ and $R_c \succ 0$ weighting matrices: a weighted sum of state error and control effort (we write Q_c, R_c in order to avoid confusion with the rotation matrix R). The input applied during Δt is

$$\mathbf{u} = \mathbf{u}_h - K(\boldsymbol{\xi} - \boldsymbol{\xi}_{\text{rand}}). \tag{4.42}$$

Observation 4.5.1

In general, neither $\boldsymbol{\xi}_{\text{near}}$ nor $\boldsymbol{\xi}_{\text{rand}}$ is an equilibrium of the system, hence the linear model is only **local**, and the controller only steers the robot toward $\boldsymbol{\xi}_{\text{rand}}$ during the time Δt (it does not stabilize it there).

With this steering the tree grows along feasible, controlled motions and reaches the goal region. Each branch is a trajectory of the model, together with its inputs.

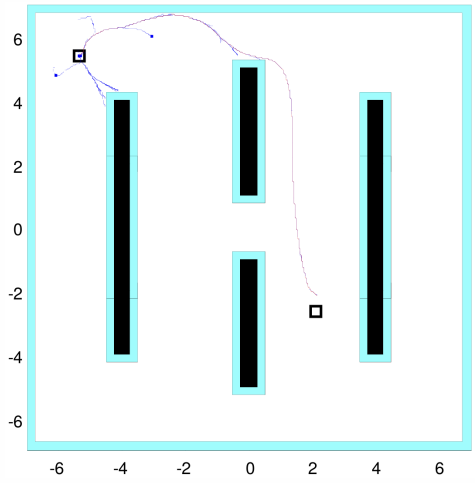


Figure 4.16: Control-based RRT in the same environment: the tree grows along feasible, controlled motions and reaches the goal region; the final trajectory is in red.

4.5.5 Remarks on the Kinodynamic RRT

- **Metric**: the Euclidean distance in the state space is a poor estimate of how hard it is to reach a state from another one. A better choice is an estimate of the *cost-to-go*, for example the one given by the LQR of the linearized model (see Glassman and Tedrake).
- **Completeness**: the extension with a fixed time step and with the best of a set of inputs is **not** probabilistically complete in general (Kunz and Stilman). Probabilistic completeness is recovered by choosing random inputs and random durations (Kleinbort et al.).
- **Optimality**: the RRT returns the first trajectory found. Asymptotically optimal variants (kinodynamic RRT*, Webb and van den Berg) rewire the tree.

4.6 Comparison and Conclusions

The two approaches can be compared as follows.

	visibility + flatness	**kinodynamic RRT**
planning space	flat outputs ($\mathbb{R}^3$ for the roadmap)	state space $\boldsymbol{\xi} \in \mathbb{R}^{12}$
dynamics	through flatness, smoothing and time scaling	simulated at each step
feasibility	from $\mathbf{p} \in C^4$, $\psi \in C^2$ and time scaling	by construction
output	smooth trajectory and inputs from Ψ	trajectory and saved inputs
main drawback	fixed homotopy class of the geometric path	slow without good steering and metric

Further remarks:

- The prior geometric path of the visibility-based approach restricts the trajectory to a given homotopy class, which may not contain a globally optimal (or even feasible) trajectory.
- A naive implementation of the RRT performs worse. Enhancements may be achieved by tuning Δt, ϵ_0 and the number of random inputs, by using a control-based approach, and by tuning the weights Q_c, R_c of the LQR or by using any other controller (even a nonlinear one).

Take-home messages:

- Quadrotors are second order and underactuated: geometric paths might not be tracked exactly.
- Flatness: we plan (x, y, z, ψ), and the state and the inputs follow algebraically; $\mathbf{p} \in C^4$ and $\psi \in C^2$ give continuous inputs.
- Decoupled approach: geometric roadmap (few nodes) $\rightarrow$ path $\rightarrow$ minimum-snap spline $\rightarrow$ time scaling for the input bounds.
- Kinodynamic RRT: feasible by construction, but slow without a good steering method and a good metric.